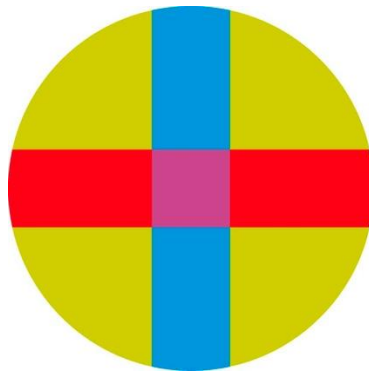


UNIVERSIDAD SAN PABLO - CEU

ESCUELA POLITÉCNICA SUPERIOR

GRADO EN INGENIERÍA DE SISTEMAS DE INFORMACIÓN



TRABAJO FIN DE GRADO

Ideación, prototipado e implementación de una aplicación web

***Ideation, prototyping and implementation
of a web application***

Autor: Jorge Ángel Vázquez Fernández

Tutor: Juan Ignacio Sanz Fuentes

Junio 2025



UNIVERSIDAD SAN PABLO-CEU

ESCUELA POLITÉCNICA SUPERIOR

División de Ingeniería

Calificación del Trabajo Fin de Grado

Datos del alumno

NOMBRE: JORGE ÁNGEL VÁZQUEZ FERNÁNDEZ

Datos del Trabajo

TÍTULO DEL PROYECTO: Ideación, prototipado e implementación de una aplicación web

Tribunal calificador

PRESIDENTE:

FDO.:

SECRETARIO:

FDO.:

VOCAL:

FDO.:

Reunido este tribunal el ____/____/_____, acuerda otorgar al Trabajo Fin de Grado presentado por D./Dña. _____ la calificación de _____

Resumen

Este TFG se centra en crear una aplicación web que facilite el transporte de mascotas en tren o avión, conectando a dueños de animales con personas que pueden acompañarlas durante el viaje. La aplicación ofrece a los usuarios la opción de registrarse como clientes o acompañantes, permitiendo así el acceso a diversas funcionalidades. Los clientes tienen la opción de solicitar el servicio indicando los detalles del animal, y los acompañantes pueden aceptar o rechazar esas solicitudes. PetTravelBuddy es una alternativa para que los propietarios de mascotas no necesiten viajar al destino junto a ellas.

Para la gestión del proyecto desde las etapas de investigación a la implementación se han utilizado las metodologías de diseño de la experiencia de usuario y design thinking manteniendo siempre un enfoque centrado en el usuario.

Se han incorporado herramientas de geolocalización para visualizar a los acompañantes próximos en un mapa, con detalles sobre su precio exigido y calificación (valoraciones). Además, se ha establecido un sistema de chat en tiempo real para que las dos partes puedan interactuar antes y durante el traslado.

La verificación de identidad se lleva a cabo a través de JWT (JSON Web Token) y se emplea MongoDB Atlas como base de datos para administrar usuarios, solicitudes y matches. Además, habrá un panel de administración para manejar usuarios, moderar situaciones e imponer bloqueos temporales. La plataforma igualmente envía correos de notificación a los usuarios cuando no están en línea, asegurando así una comunicación eficiente en todo momento.

Palabras Clave

Transporte, mascota, acompañante, cliente, administrador, tren, avión, chat, ruta, notificación, mongo, bloqueo, autenticación, precio, dashboard, ux, design thinking, web.

Abstract

This TFG focuses on creating a web application that facilitates the transportation of pets by train or plane, connecting pet owners with people who can accompany them during the journey. The application offers users the option to register as clients or companions, thus gaining access to different functionalities. Clients can request the service by providing details about the pet, and companions can accept or reject those requests. PetTravelBuddy serves as an alternative for pet owners who do not need to travel to the destination along with their pets.

For the comprehensive management of the project, from the research stages to implementation, user experience design methodologies and design thinking have been used, always maintaining a user-centered approach.

Geolocation tools have been incorporated to display nearby companions on a map, including information about their required price and rating (reviews). Additionally, a real-time chat system has been implemented to allow both parties to interact before and during the transfer.

Identity verification is carried out through JWT, and MongoDB Atlas is used as the database to manage users, requests, and matches. Furthermore, there will be an admin panel to manage users, moderate situations, and impose temporary bans. The platform also sends email notifications to users when they are offline, ensuring effective communication at all times.

Keywords

Transport, pet, companion, client, administrator, train, plane, chat, itinerary, notification, mongo, block, authentication, price, dashboard, ux, design thinking, web.

Índice de contenidos

Capítulo 1 Introducción	1
1.1 Contexto del TFG	1
1.2 Objetivos.....	1
1.3 Organización del trabajo.....	2
Capítulo 2 Gestión del proyecto	5
2.1 Modelo de ciclo de vida.....	5
2.2 Planificación.....	6
2.3 Estimación de costes.....	17
Capítulo 3 Análisis.....	18
3.1 Especificación de requisitos.....	18
3.2 Análisis de los Casos de Uso	20
3.3 Análisis de seguridad	22
Capítulo 4 Diseño e implementación	23
4.1 Arquitectura del sistema	23
4.1.1 Arquitectura física	23
4.1.2 Arquitectura lógica	24
4.2 Diseño de datos	24
4.3 Diseño de la interfaz de usuario	26
4.4 Colecciones de documentos	29
4.5 Entorno de construcción.....	30
4.6 Referencia al repositorio de software	32
Capítulo 5 Resultados	33
5.1 Plan de pruebas	33
5.2 Aplicación desarrollada.....	33
Capítulo 6 Conclusiones y líneas futuras	36
6.1 Conclusiones	36
6.2 Líneas futuras.....	37
Bibliografía.....	38

Anexo I – Empathy Map	40
Anexo II – User Persona.....	41
Anexo III – Journey Map	42
Anexo IV – Estimación de costes	43
Anexo V – Wireframe	51
Anexo VI – Colecciones.....	52

Índice de figuras

Ilustración 1. Los cinco planos de la experiencia de usuario. The Elements of User Experience de Jesse James Garrett	2
Ilustración 2. Capas de la experiencia de usuario desde lo abstracto hasta lo concreto. The Elements of User Experience de Jesse James Garrett.....	4
Ilustración 3. Fases del Design Thinking. Basado en el enfoque de la d.school (Stanford University).....	5
Ilustración 4. User Persona perfil Jorge.....	13
Ilustración 5. User Flow Cliente. Elaboración propia con Figma	14
Ilustración 6. User Flow Acompañante. Elaboración propia con Figma	14
Ilustración 7. User Flow Administrador. Elaboración propia con Figma	15
Ilustración 8. Diagrama de caso de uso: Buscar acompañante. Elaboración propia con Draw.io	20
Ilustración 9. Diagrama de caso de uso: Solicitar acompañamiento. Elaboración propia con Draw.io.....	21
Ilustración 10. Diagrama de caso de uso: Chatear con el acompañante, Elaboración propia con Draw.io	21
Ilustración 11. Diagrama Entidad-Relación. Elaboración propia con dbdiagram.io	26
Ilustración 12. Wireframe Cliente. Elaboración propia con Figma.	27
Ilustración 13. Wireframe Acompañante. Elaboración propia con Figma.	28
Ilustración 14. Wireframe Administrador. Elaboración propia con Figma.	28
Ilustración 15. Colecciones de MongoDB.....	29
Ilustración 16. Documento de ejemplo de la colección users en MongoDB.	30
Ilustración 17. Flujo de pantalla cliente. Elaboración propia con Miro	34

Ilustración 18. Flujo de pantalla acompañante. Elaboración propia con Miro	34
Ilustración 19. Flujo de pantalla administrador. Elaboración propia con Miro	35
Ilustración 20. Empathy Map. Elaboración propia con Figma	40
Ilustración 21. Extensión User Persona Jorge	41
Ilustración 22. Journey Map. Elaboración propia con Figma	42
Ilustración 23. Selección del tamaño de la aplicación en la herramienta Estimate My App.	43
Ilustración 24. Selección nivel interfaz de usuario en la herramienta Estimate My App.	44
Ilustración 25. Selección usuarios y cuentas en la herramienta Estimate My App.	44
Ilustración 26. Selección contenido generado por el usuario en la herramienta Estimate My App.	45
Ilustración 27. Selección localizaciones/fechas y social en la herramienta Estimate My App.....	46
Ilustración 28. Selección facturación y eCommerce en la herramienta Estimate My App.	47
Ilustración 29. Selección admin, feedback y análisis en la herramienta Estimate My App.	48
Ilustración 30. Selección APIs y seguridad en la herramienta Estimate My App.	49
Ilustración 31. Rango de precios por hora estimado para el desarrollo de aplicaciones según Cleveroad (2024).....	50
Ilustración 32. Wireframe Inicio. Elaboración propia con Figma.	51
Ilustración 33. Documento de ejemplo de la colección solicitudes en MongoDB.	52
Ilustración 34. Documento de ejemplo de la colección reportes en MongoDB.	52
Ilustración 35. Documento de ejemplo de la colección matches en MongoDB.	53

Ilustración 36. Documento de ejemplo de la colección chats en MongoDB. 53

Ilustración 37. Documento de ejemplo de la colección mensajes en MongoDB..... 54

Índice de tablas

Tabla 1. Requisitos no funcionales 19

Capítulo 1

Introducción

1.1 Contexto del TFG

Jorge es el dueño de un perro Bichón Maltés que necesita viajar. Necesita un servicio en el que confiar para trasladar a su mascota en avión o tren sin preocuparse por los trámites y sin tener que viajar con él. Actualmente, por ejemplo, en Renfe, no existe un servicio específico de acompañamiento para animales como los que hay para niños que viajan solos, lo que puede generar incertidumbre y estrés en los dueños de mascotas. Es por todo ello que se me ocurrió esta idea para la realización de la aplicación web para mi TFG.

En este TFG se aplicarán metodologías de gestión de proyectos centrados en usuario. Para ello se emplearán elementos/planos de la experiencia de usuario que dirán qué hacer y Design Thinking que dirá como hacerlo, con el objetivo de construir una experiencia de usuario clara y centrada en necesidades reales.

1.2 Objetivos

Hay tres claros objetivos con la realización de este TFG. El primero es el académico con el cual se pretende demostrar habilidades con las distintas tecnologías. El segundo es el objetivo de la aplicación en sí, la cual tendrá por objetivo ayudar a los dueños de las mascotas a trasladar de forma segura y sin problemas a sus animales, ya sea en tren o en avión, hasta un destino fijado sin tener que viajar con ellas. Para ello, y gracias a esta web, se podrá contactar con un acompañante que se encargue del papeleo y asegure que la mascota llega al transporte, así como entregado en el destino.

Esta idea de transporte de mascotas permitirá a los usuarios contactar con un acompañante para el traslado de forma segura y eficiente para las mismas. Los afectados serán los dueños de mascotas que necesitan enviar a sus animales a un destino de España sin acompañarlos y, por otra parte, los acompañantes que buscan una oportunidad de ingresos transportándolos. Por otra parte, los beneficiados serán los usuarios ya que se ofrece un sistema confiable similar a Uber, donde acompañantes que van a viajar a otros destinos, pueden aceptar solicitudes y encargarse del traslado.

Por otro lado, el tercer objetivo se trata de lo personal, es decir, aprender, crecer y explorar nuevas tecnologías en un entorno real. Este proyecto ha supuesto una oportunidad para profundizar en áreas como el diseño centrado en el usuario, el desarrollo web o el uso de bases de datos como MongoDB. Más allá de entregar una solución útil, la intención también ha sido poner a prueba mis conocimientos, habilidades, metodologías y tecnologías como Tailwind CSS, Node, Vercel, etc.

1.3 Organización del trabajo

Para la organización del proyecto, se utilizarán los 5 planos de la experiencia de usuario (UX).

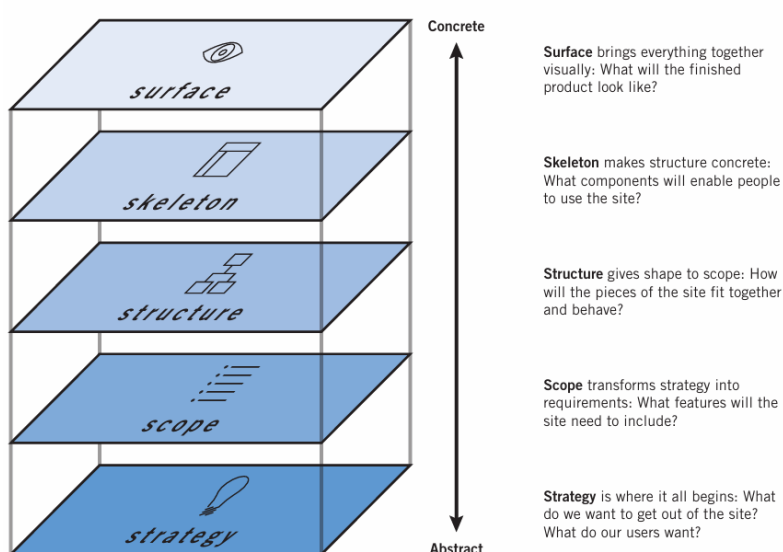


Ilustración 1. Los cinco planos de la experiencia de usuario. *The Elements of User Experience* de Jesse James Garrett

En esta imagen se pueden ver los distintos planos de la experiencia de usuario (UX). Cada uno, responde a una pregunta distinta. A rasgos generales, estos planos son describen lo siguiente:

- **Estructura:** En esta fase se define la razón de ser del proyecto, se responde a la pregunta ¿qué queremos obtener del sitio?, es decir, se trata de comprender qué se espera obtener con la aplicación web y cuáles son las verdaderas necesidades de los usuarios.
- **Alcance:** Una vez clara la estrategia hay que responder a la pregunta ¿qué características deberá incluir el sitio?, así que el siguiente paso es traducirla en requisitos concretos. Aquí se decide qué funcionalidades debe tener la web y qué tipo de contenidos ofrecerá. Se empieza a delimitar qué se incluirá y qué no, priorizando aquello que aporte más valor.
- **Estructura:** La estructura organiza cómo se relacionarán entre sí todas las piezas del sitio. No se trata solo de decidir qué habrá, sino cómo se conectará y comportará todo dentro de una misma lógica.
- **Esqueleto:** En este plano se va a responder a la pregunta ¿qué componentes permitirán a las personas usar la aplicación? Se diseña la distribución de los elementos en pantalla: botones, formularios, menús, jerarquías visuales... Aquí se decide cómo se disponen las cosas para que el usuario pueda utilizarlas sin complicaciones.
- **Superficie:** Finalmente, la superficie da forma visual a todo lo anterior, ¿cómo se verá el producto final? Es lo que el usuario ve cuando entra en la web: los colores, las tipografías, los estilos, la estética general.

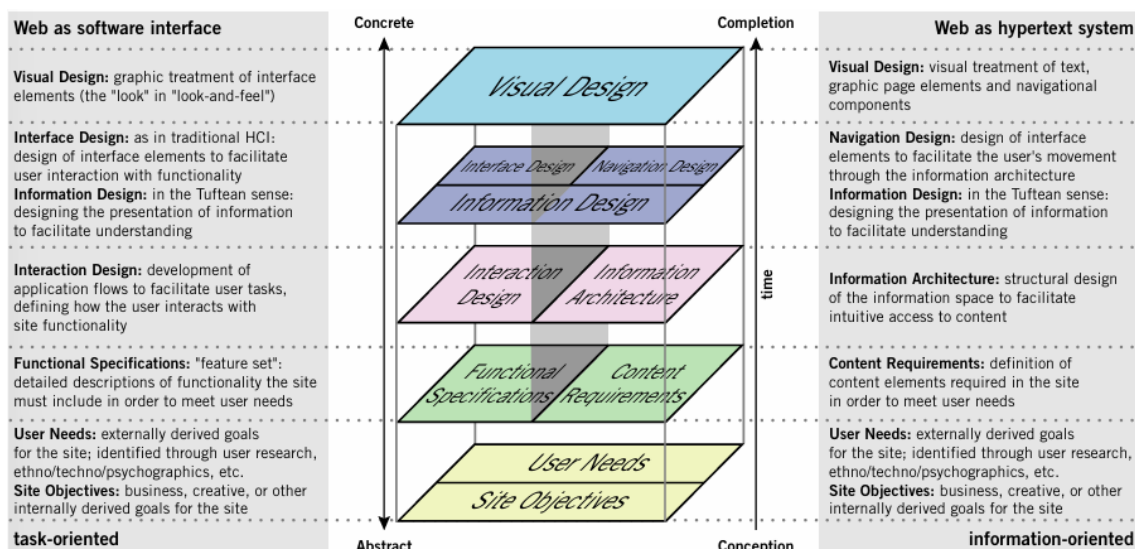


Ilustración 2. Capas de la experiencia de usuario desde lo abstracto hasta lo concreto. *The Elements of User Experience* de Jesse James Garrett

Una web no solo está orientada a contenido, sino que también actúa como una interfaz interactiva, similar a una aplicación. Por eso, diseñar una buena experiencia de usuario implica mucho más que elegir colores bonitos o una tipografía atractiva.

Todo parte de la base: entender qué necesitan los usuarios y qué objetivos tiene el sitio. Sobre esa base se construyen los requisitos (qué debe ofrecer la web y cómo debe hacerlo), y a partir de ahí se definen decisiones clave como la organización del contenido (arquitectura de la información), los caminos que recorrerá el usuario (diseño de interacción) y cómo se mostrará todo eso de forma clara e intuitiva (diseño de interfaz y navegación).

Solo al final se llega a lo más visible: el diseño visual. Pero todo lo anterior tiene que estar bien hecho para sacar el máximo partido a la experiencia de usuario.

Adicionalmente, se utilizará la metodología de design thinking para obtener información del usuario y progresar en cada etapa:

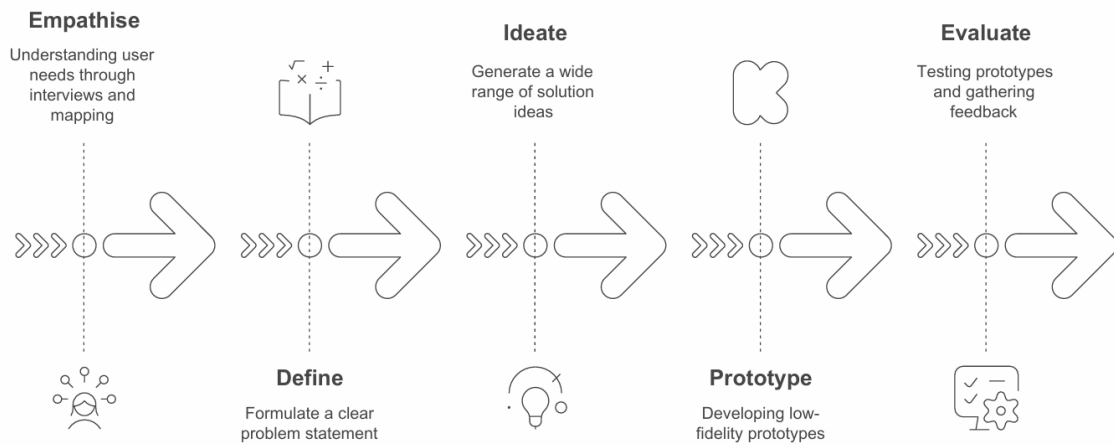


Ilustración 3. Fases del Design Thinking. Basado en el enfoque de la d.school (Stanford University).

Capítulo 2

Gestión del proyecto

2.1 Modelo de ciclo de vida

Una metodología de desarrollo define cómo se estructura y organiza el proceso de construcción del software (fases, entregas, planificación, etc.).

Para el desarrollo de este proyecto se ha seguido un modelo de ciclo de vida incremental. Esta es una metodología de desarrollo de software que divide el proyecto en incrementos o partes más pequeñas, donde cada incremento entrega una porción funcional del producto final.

El sistema se ha construido por fases, comenzando por los elementos más esenciales como el sistema de autenticación (registro, login), para posteriormente incorporar

funcionalidades como la visualización de acompañantes en un mapa con geolocalización real, la creación y gestión de solicitudes de acompañamiento para mascotas, y finalmente, la mensajería en tiempo real mediante un chat y el panel de administración para la gestión del sistema.

Cada uno de estos módulos se ha diseñado, implementado y probado antes de pasar al siguiente, permitiendo así detectar errores y adaptar los requisitos en función del progreso. Este enfoque también ha favorecido el uso de tecnologías, como WebSockets para el sistema de chat o Leaflet para la visualización del mapa de acompañantes.

2.2 Planificación

Antes de comenzar con la parte técnica del desarrollo de la aplicación web, era esencial dedicar un tiempo considerable a comprender bien las necesidades tanto del usuario como del sistema, además de llamar e informarme de los servicios que proporcionan las distintas empresas competidoras. En esta fase preliminar, el enfoque estuvo en la realización de distintos diagramas como los User Flow, Journey Map, Casos de Uso, etc. También realicé un análisis de la competencia y un Google Forms para conocer las opiniones de los usuarios. Todo esto, permitió construir una visión global y detallada del funcionamiento deseado. Este análisis inicial, aunque pueda parecer alejado del código, ha sido clave para garantizar que las funcionalidades que más tarde se implementarían estuviesen en línea con lo que se espera del producto final. Todo esto llevó un tiempo estimado de mes y medio.

Una vez hecho esto, se pasó a planificar el desarrollo completo del proyecto. Para ello se empezó a desarrollar la aplicación a partir de lo que generó de primeras la herramienta Lovable. Luego, se organizaron las tareas en fases o requisitos que abarcaban desde la creación del backend y la base de datos (en MongoDB), hasta el diseño de la interfaz web (frontend), la integración de un chat en tiempo real, la geolocalización de acompañantes en el mapa, el sistema de solicitudes entre usuarios y el panel de administración (dashboard). Esta etapa inicial tuvo una duración de mes y medio.

En cuanto a los hitos, algunos de los principales fueron:

- Poder contactar con todas las empresas y obtener la información que quería
- Hacer diagramas verdaderamente útiles para facilitarme el desarrollo de la aplicación web.
- Implementación del mapa con localización real de acompañantes, así como de rutas.
- Integración del chat entre usuarios.

Para la realización de este TFG, se han usado dos metodologías de diseño:

A. Elementos de la experiencia del usuario: Esta metodología se centra en diseñar productos que ofrezcan una experiencia de uso satisfactoria, intuitiva y útil para el usuario. Su objetivo es comprender las necesidades de los usuarios y adaptar el diseño de la interfaz y navegación para que el uso de la plataforma sea sencillo y útil. Esto se puede conseguir mediante la elaboración de User Flows, Journey Maps...

Esta metodología cuenta con 5 planos:

- 1. Estrategia:** Aquí se van a definir los objetivos del sistema y las necesidades de los usuarios.
 - a) Propósito del sistema:** Crear un servicio para facilitar el transporte de mascotas en trenes y aviones, permitiendo a los dueños solicitar un acompañante que gestione la recogida y entrega en estaciones y aeropuertos, así como el papeleo.
 - b) Necesidades de los usuarios:** Por un lado, están los dueños de mascotas que necesitan trasladar a sus animales sin preocupaciones, y por otro lado acompañantes que quieren sacarse un dinero con una herramienta sencilla y eficiente para

gestionar la recogida y entrega de las mascotas. Para dejarlo más claro, ver Anexo I

- c) Análisis de la competencia:** Para llevar a cabo este análisis se ha elaborado un documento comparativo en Excel donde se han recopilado diferentes plataformas, tanto competidores directos como indirectos, que ofrecen servicios relacionados con el transporte de mascotas o el desplazamiento de pasajeros con animales. La tabla analiza para cada competidor aspectos clave como sus características generales, nivel de accesibilidad, flujo de usuario, funcionalidades ofrecidas, precios, público objetivo y propuesta de valor. Esto permite entender qué puntos fuertes y débiles presenta la aplicación web a desarrollar en comparación con la competencia.

- 2. Alcance:** Esta fase traduce la estrategia en requisitos funcionales y de contenido, determinando qué funcionalidades tendrá la aplicación. Estos requisitos salen del journey map, empathy map y análisis de la competencia hechos anteriormente.

a) Requisitos funcionales:

- Registro y autenticación de usuarios (dueños de mascotas y acompañantes)
- Solicitud de servicio con selección de destino y detalles de la mascota.
- Asignación de acompañantes de forma manual.
- Sistema de chat entre el usuario y la persona encargada de la mascota.
- Seguimiento del estado del servicio en tiempo real.
- Historial de viajes y valoraciones de los encargados, opción de reseña.
- Opción de reportar incidencias
- Recuperación de contraseña por correo electrónico

- Notificación automática por email cuando hay cambios en el estado del servicio
- Opción de desactivar la cuenta o solicitar eliminación de datos personales

b) Requisitos de contenido:

- Normativas del transporte de mascotas
- Estado del servicio
- Preguntas frecuentes
- Política de seguridad y privacidad
- Términos y condiciones de uso del servicio
- Información sobre el proceso de acompañamiento paso a paso.
- Enlace de contacto o soporte técnico

3. Estructura: Organización de la información e interacción de los usuarios con el sistema.

a) Dueños mascotas: su flujo de navegación sería el siguiente:

- **Inicio:** Opción principal para solicitar servicio.
- **Historial de solicitudes:** Lista de servicios pasados y actuales.
- **Seguimiento en tiempo real:** Estado del traslado y comunicación con el encargado.
- **Perfil:** Datos personales, mascotas registradas y métodos de pago.

b) Encargado del traslado (acompañante): su flujo de navegación es el siguiente:

- **Panel de solicitudes:** Lista de servicios pendientes con opción de aceptar.
- **Estado de servicio:** Marcar el estado.
- **Chat con el usuario:** Comunicación directa con el dueño de la mascota.

- **Historial de servicios:** Registro de mascotas transportadas y valoraciones recibidas (puntuación, reseñas...)

c) **Administrador:** su flujo de navegación es el siguiente:

- **Dashboard general:** Resumen de actividad del sistema (solicitudes activas, finalizadas, canceladas).
- **Gestión de usuarios:** Listado de dueños de mascotas y empleados registrados, con opciones de edición (bloquear, etc).
- **Panel de incidencias:** Reportes de problemas (mascotas no entregadas, empleados con mala puntuación, etc.).
- **Reportes y estadísticas:** Análisis del uso de la plataforma, número de transportes realizados y feedback de usuarios.

4. Esqueleto:

a) **Wireframes:** Los wireframes son representaciones visuales esquemáticas de las distintas pantallas del sistema. Su objetivo es definir la estructura básica de los elementos y componentes de cada vista, sin entrar aún en aspectos gráficos como colores o imágenes finales. Permiten visualizar de forma anticipada la distribución y jerarquía de la información, facilitando así decisiones tempranas sobre usabilidad y navegación.

Este aspecto se desarrollará con más detalle en el apartado 4.3.

b) **Diseño de navegación:** Desde el principio, el proyecto se pensó para que tuviera una navegación sencilla y de esta manera facilitar el uso de la plataforma a cualquier tipo de usuario, incluso sin experiencia previa. Se implementó un

header permitiendo acceder a todas las partes de la aplicación web. Este diseño permite que el usuario siempre tenga a mano las acciones principales sin necesidad de volver atrás o realizar demasiados clics. Además, la estructura varía según el rol (cliente, acompañante o administrador), mostrando únicamente las opciones relevantes para cada tipo de usuario, lo cual mejora la claridad y evita distracciones.

- c) **Diseño de información:** El contenido mostrado en cada pantalla se organizó con el objetivo de ser visualmente claro, útil y generar confianza. Por ejemplo, en la sección donde el cliente puede seleccionar a un acompañante, se muestra una foto del usuario, su valoración, los detalles del trayecto, y una breve descripción del servicio ofrecido. Esto permite tomar decisiones y además transmite mayor confianza y seguridad.

5. Superficie: diseño, colores, tipografías, componentes estáticos e interactivos que mejoran la legibilidad, usabilidad, accesibilidad, consistencia y comprensión de la información que se muestra y cómo usarla. Algunas ideas son: Tamaño de letra ajustable, fuentes legibles con alto contraste para mejorar la visibilidad, ilustraciones de mascotas y encargados para hacer la interfaz más cercana, Web Responsive Design (si la pantalla se hace pequeña, que se siga viendo bien). Para dar forma visual al proyecto y garantizar que la interfaz se vea bien en distintos dispositivos, se ha utilizado Tailwind CSS. Gracias a él, ha sido más sencillo mantener una estética clara, ajustar tamaños de letra y asegurarse de que todos los elementos como botones o formularios se adapten correctamente tanto en móvil como en ordenador.

B. Design Thinking: Es una metodología centrada en la resolución creativa de problemas, que se basa en entender profundamente al usuario antes de proponer una solución, es decir, nos dice el cómo. En el contexto de este TFG,

design thinking ha permitido analizar el problema del traslado de mascotas desde el punto de vista de los usuarios (clientes y acompañantes), generar nuevas ideas, y validarlas mediante pruebas antes de implementarlas definitivamente.

Esta metodología se divide en 5 etapas:

1. Investigación:

- a) Entendimiento del problema:** se han realizado encuestas a gente con y sin mascotas para conocer sus preocupaciones a la hora de que estas viajen solas en tren o avión. En este Google Forms también se preguntó sobre la utilidad de esta aplicación web. Por último, se realizó un análisis de servicios existentes en aerolíneas y trenes (Iberia, Renfe) para identificar la falta de un servicio de acompañamiento puerta a puerta.
- b) Perfiles de usuario:** representan las frustraciones u objetivos de los usuarios. Para dejar esta parte clara, algunos ejemplos son:
 - Jorge es dueño de un Bichón Maltes. Jorge tiene familia en Barcelona y quiere asegurarse de que acompañen a su mascota de puerta a puerta en el avión ya que él no puede viajar por tema de trabajo. Más información sobre Jorge en Anexo II.

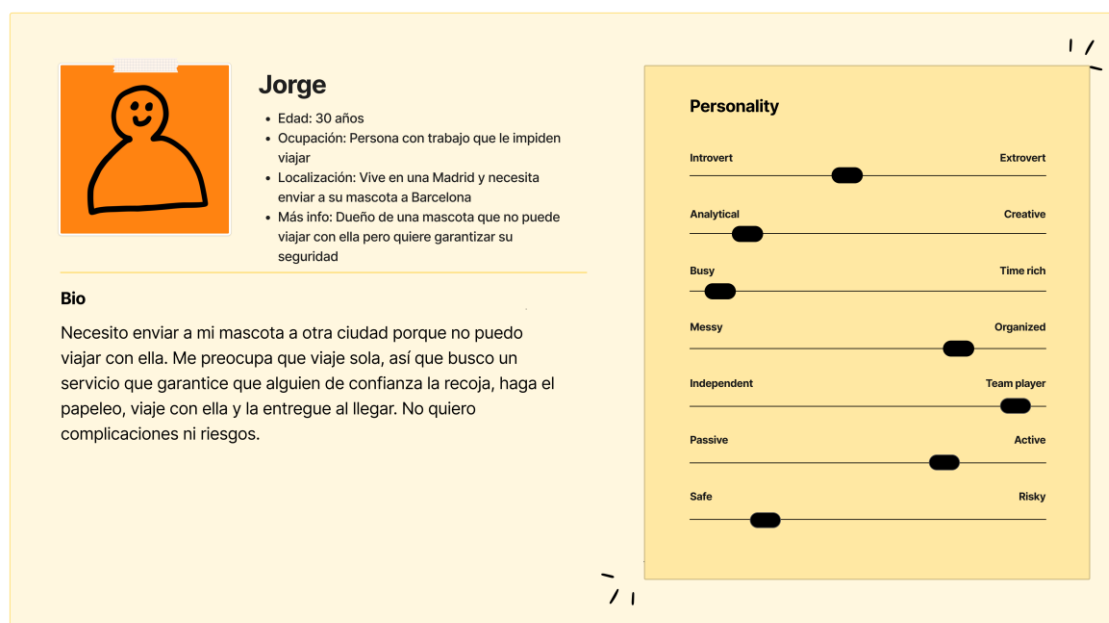


Ilustración 4. User Persona perfil Jorge

- c) **Casos de uso:** El usuario solicita un servicio de recogida de mascota. Luego, un acompañante disponible acepta la solicitud y se pone en contacto con el usuario. El usuario deja a su mascota en la zona acordada, el acompañante la recoge y la traslada al avión/tren. A continuación, el acompañante en el destino entrega la mascota a la persona indicada. Por último, El usuario evalúa la experiencia del servicio.
- d) **Journey Map:** Su objetivo principal es comprender las necesidades, emociones y oportunidades de mejora del usuario en cada etapa del proceso, desde que descubre el servicio hasta que lo finaliza. En el contexto de este TFG, se ha utilizado un journey map (ver Anexo III) para identificar cómo es la experiencia de un cliente que desea trasladar a su mascota mediante un acompañante. En él, detallan las acciones que realiza, las tareas asociadas, sus emociones en cada fase y posibles oportunidades de mejora.

2. Definir: Aquí se concreta el problema real que hay que resolver. A partir de lo aprendido en la fase anterior, se ven las necesidades centradas en el usuario.

3. Ideas:

a) **User Flow:** Es un diagrama que representa gráficamente el recorrido que sigue un usuario dentro de una aplicación o plataforma para lograr un objetivo concreto. Este tipo de representación ayuda a visualizar cómo se estructura la navegación, qué decisiones puede tomar el usuario y cómo se conectan entre sí las distintas pantallas o acciones.

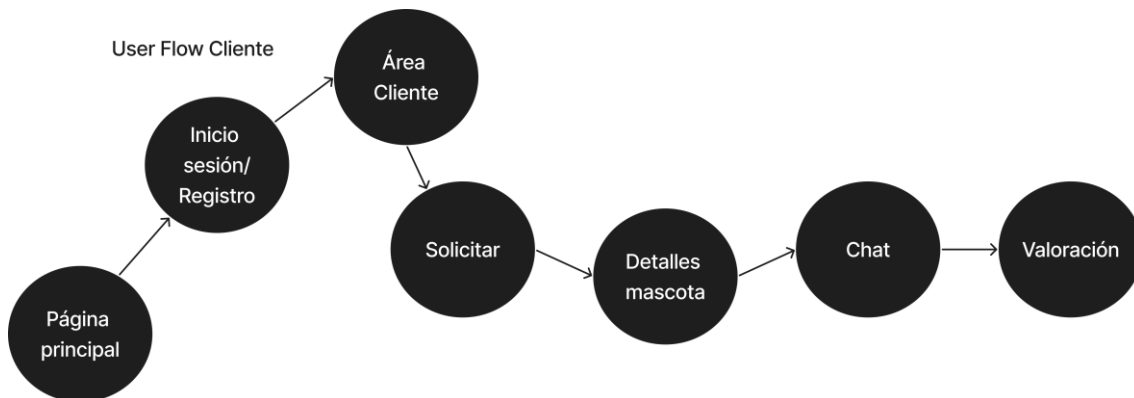


Ilustración 5. User Flow Cliente. Elaboración propia con Figma

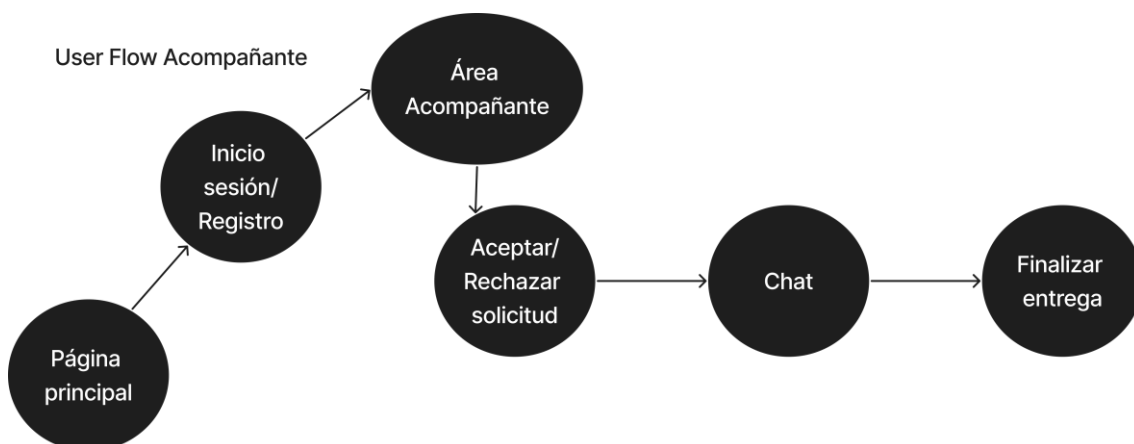


Ilustración 6. User Flow Acompañante. Elaboración propia con Figma

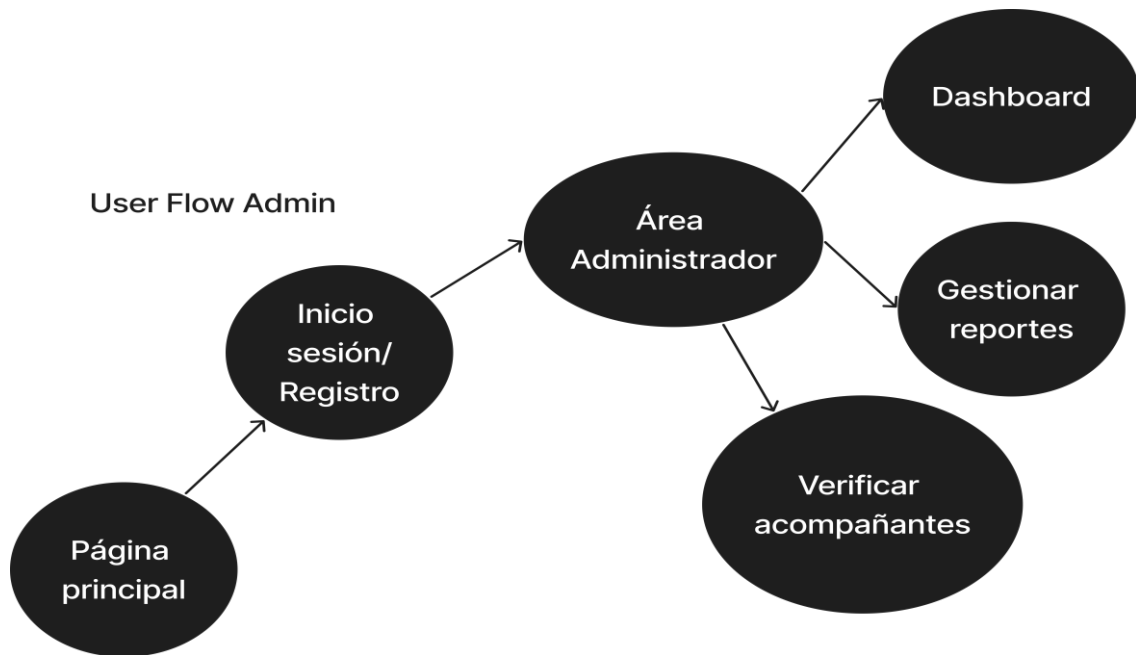


Ilustración 7. User Flow Administrador. Elaboración propia con Figma

- b) **Wireframes:** Los wireframes son representaciones visuales esquemáticas de las distintas pantallas del sistema. Su objetivo es definir la estructura básica de los elementos y componentes de cada vista, sin entrar aún en aspectos gráficos como colores o imágenes finales. Permiten visualizar de forma anticipada la distribución y jerarquía de la información, facilitando así decisiones tempranas sobre usabilidad y navegación.

Este aspecto se desarrollará con más detalle en el apartado 4.3.

4. Implementación

- a) **Prototipos:** Durante las primeras fases del desarrollo se trabajó con prototipos de interfaz de usuario que sirvieron para validar las principales funcionalidades antes de escribir código. Estos prototipos eran de calidad media e incluían pantallas como el área del cliente, el área del acompañante, los formularios de solicitud, etc. Gracias a estos prototipos pude detectar mejoras a

la hora de organizar la información, así como simplificar/quitar algunos flujos de navegación para facilitar el uso a personas sin mucha experiencia con las tecnologías.

b) Frontend y Backend: La aplicación web consta de dos grandes bloques: el back-end y el front-end.

El back-end se hizo con Node.js y Express, conectándose a una base de datos no relacional en la nube, MongoDB Atlas. También pertenecen al back-end las diferentes rutas para por ejemplo la autenticación, solicitudes, chat...

En el front-end, se creó una interfaz adaptada a cada rol (cliente, acompañante y administrador), incluyendo funcionalidades como el mapa interactivo de acompañantes cercanos, el formulario de la solicitud de acompañamiento, etc. La aplicación es funcional pudiendo ejecutarse en local.

5. Análisis/informes:

a) Usabilidad: Este apartado consiste en observar usuarios reales y aunque no se ha llevado a cabo una fase formal de pruebas con usuarios reales, durante el desarrollo se consultaron con frecuencia personas externas para verificar la comprensión de las interfaces, así como a través de un formulario. Se observó especialmente si eran capaces de navegar por la plataforma sin indicaciones, enviar solicitudes, comunicarse por el chat o encontrar acompañantes por ubicación. Gracias a esto, se pudieron hacer ajustes o replanteamientos por ejemplo en cuanto a la colocación de elementos en la aplicación web.

b) Pruebas divididas: Esta etapa consiste en comparar las mejoras introducidas. Se compararon diferentes enfoques en por ejemplo la pantalla de solicitar acompañante. Por ejemplo, se probó inicialmente con un formulario más largo dividido en secciones,

que se reemplazó finalmente por un formulario compacto, al resultar más fácil de completar.

- c) Informes de análisis:** En este apartado, se analiza el tiempo empleado por los usuarios en la aplicación o la tasa de rebote (porcentaje de usuarios que entran en una página y se van sin interactuar con ella). Dado que el sistema está pensado para ejecutarse en entorno local, no he integrado ese tipo de métricas. No obstante, en futuras versiones sería interesante incorporar herramientas como Google Analytics o informes personalizados, especialmente si se desplegara públicamente. Esto permitiría entender mejor el comportamiento de los usuarios, detectar puntos de abandono y mejorar la experiencia basándonos en esos datos.

2.3 Estimación de costes

Para estimar el coste de la aplicación web, se ha hecho uso de la herramienta online Estimate My App, que permite calcular una estimación aproximada a partir de las funcionalidades seleccionadas, el tamaño del proyecto, el tipo de interfaz, y otros aspectos clave como el sistema de autenticación, geolocalización, chat o panel de administración. Esta herramienta facilita una visualización rápida y orientativa del esfuerzo económico que implicaría llevar a producción una solución como la diseñada en este TFG.

También, se ha tenido en cuenta el blog de Cleveroad, donde se explican los distintos factores que afectan al coste de desarrollo de una aplicación: desde el número de plataformas y la complejidad funcional, hasta la experiencia del equipo de desarrollo o el modelo de negocio. Esta fuente contribuye a entender por qué el coste puede variar significativamente y qué elementos deben valorarse en el planteamiento económico de un producto digital.

El coste se estima en \$41850, es decir, 36880€ y la duración del proyecto en unas 17 semanas. Para ver más detalles sobre todo esto, consultar Anexo IV.

Capítulo 3

Análisis

3.1 Especificación de requisitos

Durante el desarrollo de PetTravelBuddy, se ha prestado especial atención a la protección de los datos personales de los usuarios, siguiendo los principios establecidos por el Reglamento General de Protección de Datos (RGPD) europeo.

La aplicación recopila únicamente los datos necesarios para prestar el servicio (nombre, ubicación, contacto, información de la mascota y datos del acompañamiento) y se informa de su uso en la sección de Política de Privacidad. En ningún caso se comparten datos con terceros sin el consentimiento explícito del usuario, salvo obligación legal.

Además, se ha implementado una Política de Cookies accesible para todos los usuarios, detallando los tipos de cookies utilizadas y permitiendo su gestión, tal y como exige la normativa. El usuario podrá ejercer sus derechos de acceso, rectificación o eliminación de sus datos, por ejemplo, escribiendo directamente a la dirección de contacto del servicio.

En definitiva, PetTravelBuddy asegura la transparencia, el consentimiento y el control por parte del usuario en todo momento. Ahora se procede a definir algunos de los requisitos no funcionales:

Tabla 1. Requisitos no funcionales

Renf.	Descripción
RNF01	La plataforma debe ser capaz de gestionar varias solicitudes
RNF02	El tiempo de respuesta del sistema no debe superar 5 segundos en operaciones clave
RNF03	Debe ser escalable para soportar más clientes a futuro
RNF04	Los datos de los usuarios y empleados deben ser almacenados y transmitidos cifrados (contraseñas)
RNF05	La plataforma debe implementar autenticación
RNF06	El administrador deberá verificar que los datos del acompañante sean reales
RNF07	Se debe minimizar el número de clics necesarios para solicitar un servicio, con una navegación clara e intuitiva
RNF08	Debe ser compatible con múltiples navegadores
RNF09	Los usuarios deben recibir notificaciones sobre cambios en el estado del servicio de su mascota por correo
RNF10	La interfaz debe adaptarse correctamente a dispositivos móviles, tablets y ordenadores (Responsive Design)
RNF11	Las contraseñas deberán tener una longitud mínima de 8 caracteres y cumplir criterios de seguridad (mayúsculas, minúsculas, número y símbolo).
RNF12	El sistema debe contar con mensajes de error y confirmación claros y accesibles para todos los roles de usuario.

RNF13	La plataforma cumple con las pautas WCAG, asegurando navegación accesible mediante estructuras semánticas, atributos aria-*, etc.
RNF14	Debe cumplir con el RGPD y normativas de protección de datos, permitiendo a los usuarios solicitar la eliminación de su información.

3.2 Análisis de los Casos de Uso

Un caso de uso es una forma de describir, de manera clara y estructurada, cómo interactúan los usuarios con un sistema para lograr un objetivo concreto. Nos ayuda a entender qué acciones realiza una persona dentro de la plataforma, qué espera conseguir y qué pasos sigue para hacerlo.

Cada caso de uso representa una situación real de uso del sistema, como por ejemplo en mi caso, solicitar acompañamiento, chatear con el acompañante o buscar acompañante, entre otros. Al representarlo en un diagrama, se identifican los actores implicados (quién lo hace) y las funcionalidades del sistema que están involucradas.

Utilizar este tipo de diagramas facilita el diseño, ya que permite visualizar el funcionamiento general de la aplicación desde la perspectiva del usuario, sin necesidad de entrar aún en detalles técnicos.

1. Caso de uso de buscar acompañante:

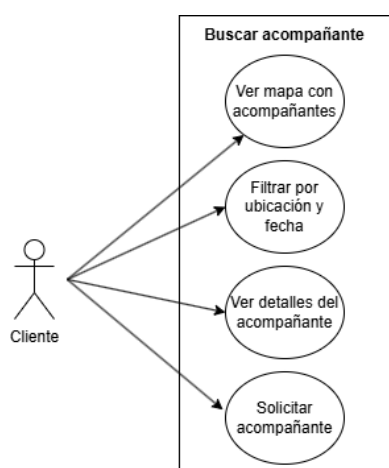


Ilustración 8. Diagrama de caso de uso: Buscar acompañante. Elaboración propia con Draw.io

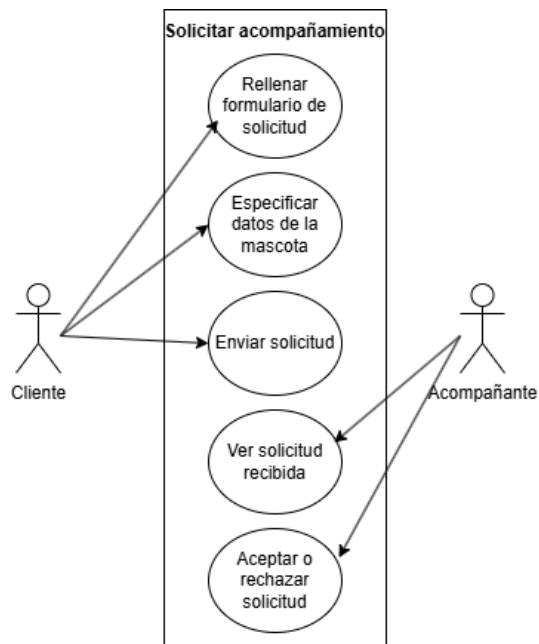


Ilustración 9. Diagrama de caso de uso: Solicitar acompañamiento. Elaboración propia con Draw.io

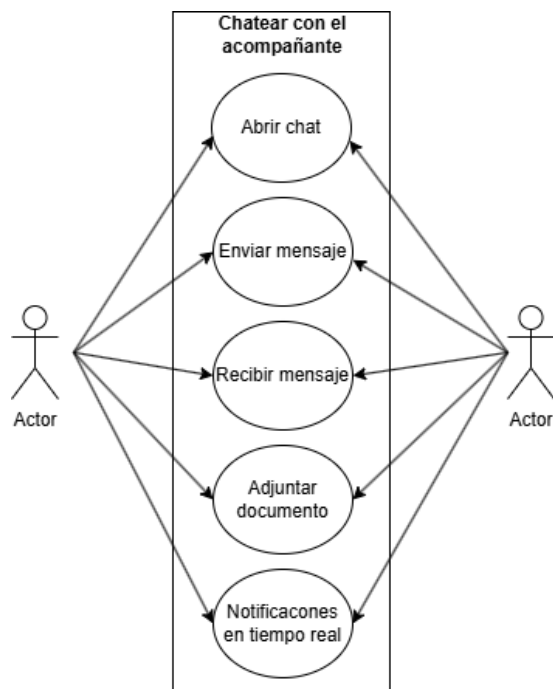


Ilustración 10. Diagrama de caso de uso: Chatear con el acompañante, Elaboración propia con Draw.io

3.3 Análisis de seguridad

En cuanto a la autenticidad, se garantiza que los usuarios son quienes dicen ser. Esto se consigue con el registro, login y verificación de DNI por parte del administrador.

La confidencialidad consiste en evitar que personas no autorizadas accedan a información sensible. En este caso, las contraseñas están cifradas y los mensajes en el chat solo pueden ser leídos por las partes involucradas.

La integridad es asegurar que la información no se modifica de forma no autorizada o accidental. Para ello, las solicitudes, valoraciones y mensajes, entre otras cosas, quedan registrados tal como fueron enviados, sin que puedan ser manipulados por terceros, únicamente el administrador puede modificarlas.

En cuanto a la disponibilidad, el sistema debe estar operativo cuando los usuarios lo necesiten. Al estar desplegado en Vercel y Render, el servicio estará disponible siempre.

Por último, la trazabilidad es poder registrar quién hizo qué, cuándo y desde dónde. Cada solicitud y cada mensaje queda asociado al usuario que lo creó (hay asociado un id a cada usuario) y por lo tanto se puede saber quién hizo cada cosa y cuando.

Capítulo 4

Diseño e implementación

4.1 Arquitectura del sistema

Para comprender el funcionamiento general del TFG, se ha definido una arquitectura del sistema que describe los diferentes componentes que hay, sin entrar en detalles de lenguajes de programación, por ejemplo.

Este tipo de arquitectura se subdivide en diversas capas o bloques funcionales: por un lado, están las interfaces utilizadas por los usuarios (clientes, acompañantes y administradores) y, por otro lado, los servicios proporcionados por el sistema como la comunicación por chat y la localización de acompañantes. Todos utilizan una capa de datos en la que se almacena información sobre viajes (solicitudes), mascotas, usuarios y mensajes.

Este enfoque en capas mejora la facilidad de mantenimiento y entendimiento además de proporcionar una visión general del sistema.

4.1.1 Arquitectura física

La arquitectura física representa los elementos presentes en la implantación y ejecución de la plataforma, es decir, lo que aparece a nivel de infraestructura. En este caso, se trata de una plataforma que utiliza un modelo web, es decir, el usuario accederá a la misma desde su dispositivo, a través de un navegador web.

El flujo general que se realiza es el siguiente: el cliente o la persona que acompaña accederá a la plataforma desde su dispositivo, que se conecta a un servidor que es el

que recibe, procesa las peticiones, gestiona los chats, comprueba los usuarios y sirve la interfaz. Este servidor también está enlazado a una base de datos (MongoDB), en la cual se guarda la información relevante del sistema. Todas estas comunicaciones se realizan a través de Internet, de manera accesible desde cualquier punto.

4.1.2 Arquitectura lógica

La plataforma se ha estructurado siguiendo una arquitectura en tres capas, un modelo muy utilizado debido a su simplicidad, claridad y facilidad. Esta forma de organizarlo permite separar las funcionalidades del sistema, facilitando tanto su desarrollo como su futura escalabilidad.

Las tres capas principales son:

- **Capa de presentación:** Es la encargada de la interacción con el usuario. Incluye las interfaces que utilizan tanto los clientes como los acompañantes para enviar solicitudes, visualizar el mapa, buscar acompañantes o usar el chat. Esta capa muestra los datos de forma amigable y recoge las acciones del usuario.
- **Capa de negocio:** Es el núcleo funcional del sistema. Aquí se gestionan las reglas internas, como verificar acompañantes, controlar reportes, o decidir qué acompañantes mostrar según la ubicación. Esta capa actúa como intermediaria entre lo que el usuario quiere hacer y lo que realmente se puede hacer (básicamente es el administrador).
- **Capa de acceso a datos:** Se encarga de comunicarse con la base de datos, almacenando y recuperando la información necesaria: datos de usuarios, mascotas, viajes, mensajes del chat, etc. Esta capa garantiza que toda la información esté organizada, segura y accesible cuando el sistema la necesita.

4.2 Diseño de datos

Voy a usar diagrama entidad-relación ya que queda todo más claro, aunque se esté usando la base de datos MongoDB que es NoSQL, orientado a documentos. Más adelante se añadirán ejemplos de colecciones.

El diseño de datos es una parte fundamental en el desarrollo de cualquier aplicación, ya que define cómo se estructura y organiza la información que el sistema necesita manejar. En este proyecto, la base de datos es la encargada de almacenar toda la información relacionada con los usuarios, las mascotas, las solicitudes de traslado, los mensajes del chat, entre otros elementos clave.

Para construir una base de datos sólida, se ha seguido un proceso estructurado que incluye las siguientes etapas:

1. Identificación de entidades: Primero se analizaron todos los elementos que forman parte del sistema para identificar las entidades principales, es decir, aquellos conceptos que deben representarse en la base de datos. Algunas de las entidades detectadas fueron:

- Usuario (cliente, acompañante, admin)
- Solicitud de traslado (información de la mascota)
- Chats
- Valoraciones, reseñas, finalización entrega, bloqueos

2. Establecer relaciones entre entidades:

- Un usuario puede tener varias mascotas.
- Un cliente puede enviar varias solicitudes.
- Un acompañante puede ofrecer uno o más trayectos.
- Un mensaje está relacionado con dos usuarios (emisor y receptor).

A continuación, se muestra el modelo entidad-relación para mayor claridad:

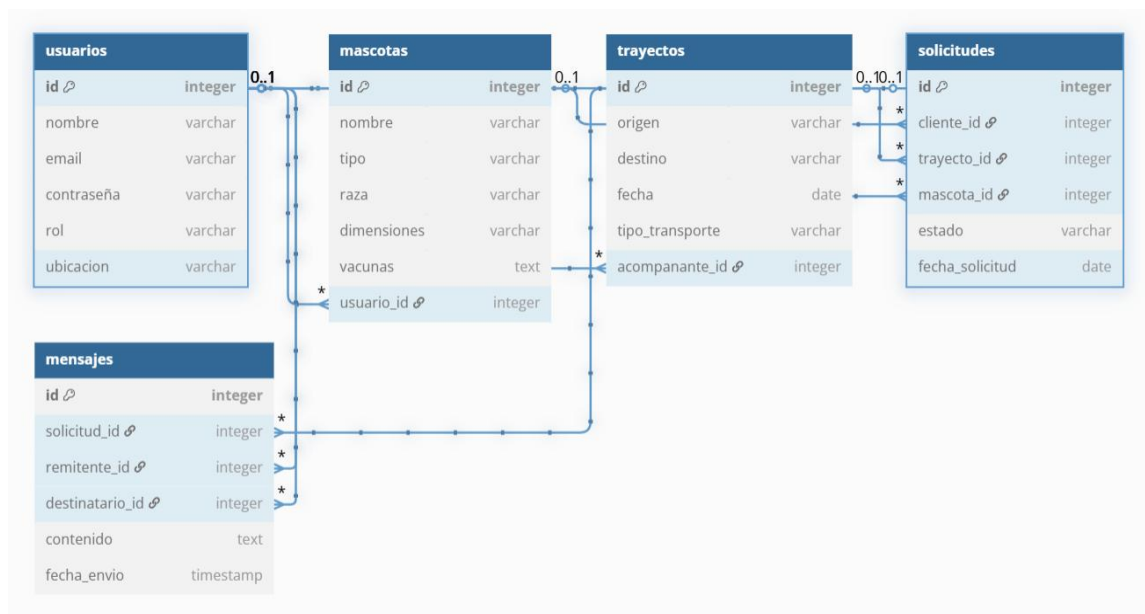


Ilustración 11. Diagrama Entidad-Relación. Elaboración propia con dbdiagram.io

Para aclarar:

- 0..1 significa cero o un registro relacionado
- 1 significa exactamente un registro relacionado
- * significa muchos registros relacionados

Por poner algún ejemplo de cómo se leería:

- **usuarios (id) → mascotas (usuario_id)**: la cardinalidad es 1->* lo que significa que un usuario puede tener muchas mascotas, pero cada mascota pertenece a un único usuario.
- **usuarios → mascotas**: la cardinalidad es 0..1->* lo que significa que un usuario puede tener muchas mascotas. Pero cada mascota solo pertenece a un usuario.

4.3 Diseño de la interfaz de usuario

Para facilitar la experiencia del usuario y garantizar una navegación intuitiva, se han diseñado distintos wireframes que representan las principales pantallas que hay, tanto para clientes como para acompañantes y administrador. Estos esquemas muestran de

forma simplificada los elementos clave de cada vista (botones, campos, menús), así como la organización de la información.

El objetivo de estos wireframes es tener una visión clara de cómo interactuarán los usuarios con la plataforma antes de realizar el diseño definitivo, aunque luego la interfaz no sea literalmente así (colocación botones, palabras...). A continuación, se muestran algunos wireframes (en Anexo V está el resto).

1. Wireframe Cliente:

Wireframe Cliente

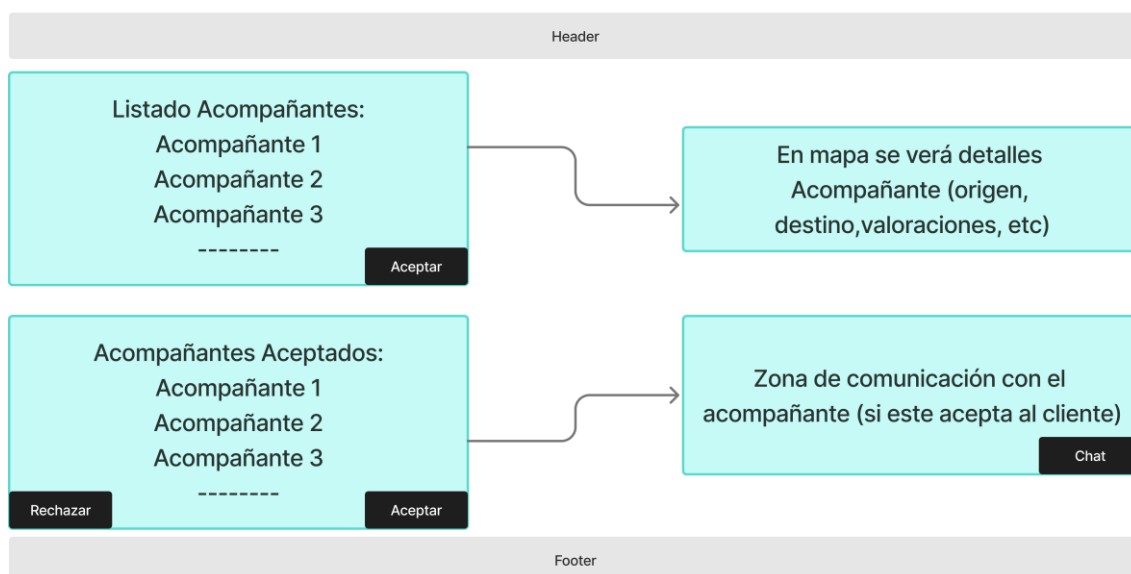


Ilustración 12. Wireframe Cliente. Elaboración propia con Figma.

En este Wireframe se pueden ver 4 zonas destacables. La primera de ellas será el listado de acompañantes, justo debajo serán los acompañantes que han aceptado la solicitud del cliente que tiene acceso a un chat para comunicarse y, por último, habrá un mapa donde salgan los acompañantes con información.

2. Wireframe Acompañante:

Wireframe Acompañante

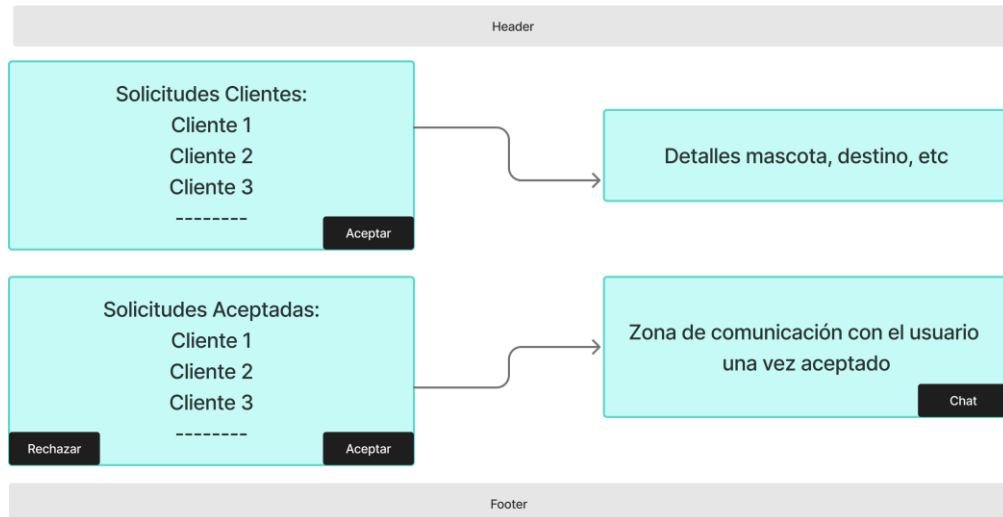


Ilustración 13. Wireframe Acompañante. Elaboración propia con Figma.

Este Wireframe cuenta con 2 zonas destacables: Las solicitudes de los clientes con la información de la mascota y los clientes que han ido aceptando con acceso a chat para comunicarse con ellos.

3. Wireframe Administrador:

Wireframe Admin

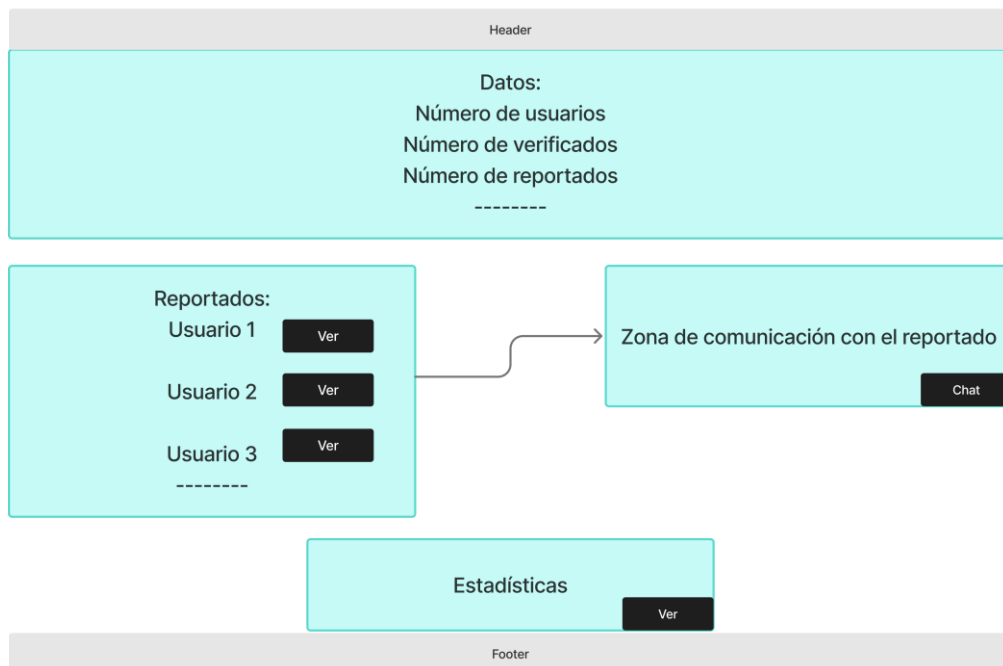


Ilustración 14. Wireframe Administrador. Elaboración propia con Figma.

Este Wireframe destaca por tener una zona de reportados/bloqueados donde el administrador puede decidir si bloquear a un usuario o no y por otro lado la zona de las estadísticas o dashboard.

4.4 Colecciones de documentos

Como se comentó anteriormente, en esta sección se analizarán las distintas colecciones que hay en el proyecto:

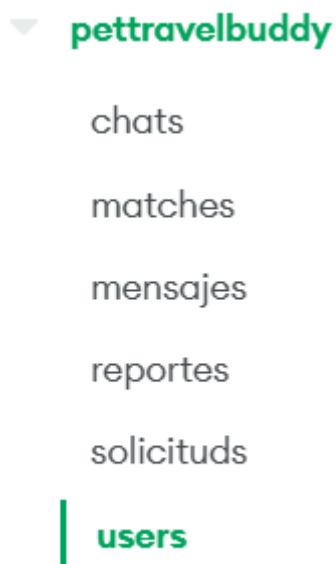


Ilustración 15. Colecciones de MongoDB

Se cuenta con 6 colecciones en MongoDB Atlas. Aquí se verá la colección users y para tener más detalle de las colecciones, ver Anexo VI.

```

_id: ObjectId('683bfa7be2b9b4a54610ba53')
nombre: "Iker"
apellidos: "López Pérez"
email: "iker@gmail.com"
password: "$2b$10$iwbVM3ADBL4WoSvDUHi20eB3sl4r89XI08B/3I00Hb6MEUkzO4500"
rol: "acompanante"
fechaNacimiento: 1998-02-17T00:00:00.000+00:00
▸ ubicacion: Object
  imagenPerfil: ""
  bio: ""
  verificado: "pendiente"
  baneado: false
  baneadoHasta: null
▸ viajes: Array (empty)
  dniFrontal: "1748761211605-DNI_Frontal.pdf"
  dniTrasero: "1748761211610-DNI_Trasero.pdf"
  createdAt: 2025-06-01T07:00:11.783+00:00
  updatedAt: 2025-06-01T07:00:11.783+00:00
__v: 0

```

Ilustración 16. Documento de ejemplo de la colección users en MongoDB.

Dado que se ha usado la base de datos MongoDB la cual es no relacional y orientado a documentos, se va a trabajar con colecciones. Una de las colecciones es la de Users, la cual está pensada para cuando se registran los usuarios. Nos indica el nombre, apellidos, email y rol entre otras cosas. La ubicación se pide una vez se inicia sesión. La biografía y la imagen de perfil pueden cambiar en un futuro si el cliente decide añadirlas. Por otro lado, si el rol es acompañante, como en este caso, deberá adjuntar tanto el DNI frontal como el trasero y mantenerse a la espera de que el administrador decida o no aceptarlo/verificarlo.

4.5 Entorno de construcción

Lo primero de todo, en cuanto al diseño y planificación, se han generado wireframes, journey maps, diagramas de casos de uso, etc apoyándose en herramientas como Draw.io, dbdiagram.io y Figma para representar visualmente la estructura y funcionamiento del sistema. Para algunas tareas relacionadas con documentación visual (como mapas de empatía), también se han utilizado herramientas como Canva o editores online que permiten crear material UX de forma rápida.

Como entorno de desarrollo, se ha utilizado Visual Studio Code, ya que soporta muchos lenguajes, hay muchas extensiones y es fácil de manejar. Ha sido desde donde se ha gestionado tanto el desarrollo del frontend como del backend.

En primer lugar, se hizo uso de la herramienta Lovable, que es una herramienta No-code que permite crear aplicaciones web a partir de una idea, para el principio del proyecto y a partir de lo que generó, se fue mejorando y añadiendo cosas. Lovable usa TypeScript/JavaScript para el frontend, Node.js para el backend y Tailwind CSS para el diseño. Usa estos lenguajes porque son ligeros, rápidos y muy usados, así como que permiten la integración de APIs externas. También se ha hecho uso de herramientas de inteligencia artificial generativas como ChatGPT como si fuera un colaborador.

El backend está escrito principalmente en JavaScript. Se ha empleado Node.js, utilizando el framework Express para estructurar el servidor y las rutas de la plataforma. La base de datos ha sido implementada con MongoDB Atlas, una base de datos NoSQL en la nube que permite trabajar con documentos en formato JSON, lo que ha facilitado el tratamiento de los datos. He optado por utilizar Mongoose como herramienta de conexión entre el servidor Node.js y la base de datos MongoDB Atlas. Mongoose actúa como una capa intermedia sobre el driver de MongoDB, permitiendo realizar validaciones de datos automáticas y simplificar las operaciones sobre las colecciones. Además, se han implementado middlewares personalizados para gestionar la autenticación (auth.js), el control de acceso (admin.js) y la presencia de usuarios (webPresence.js).

En el desarrollo del frontend, se ha utilizado React junto con TypeScript (.tsx). Además, se ha hecho uso de HTML. Los estilos están gestionados con Tailwind CSS. Es un framework que permite aplicar estilos directamente desde el HTML. También, se ha utilizado Vite como herramienta de desarrollo. Para la geolocalización y visualización en mapa, se ha integrado la librería Leaflet junto con React Leaflet, lo que ha permitido mostrar la ubicación de los acompañantes. Además, se utilizó la librería OverlappingMarkerSpiderfier para gestionar los marcadores solapados.

Por otro lado, se ha empleado socket.io para implementar el sistema de chat privado entre clientes y acompañantes, ofreciendo comunicación en tiempo real dentro de la plataforma. También se ha hecho uso de Nodemailer para el envío de correos electrónicos en determinadas acciones como el registro o la gestión de solicitudes.

Por último y en cuanto al despliegue de la aplicación web, se ha utilizado Vercel para el frontend y Render para el backend. Vercel es una plataforma que permite desplegar aplicaciones web del lado del cliente de forma rápida y sencilla. Es especialmente popular para proyectos hechos con tecnologías como React. Da su propio dominio, aunque es posible adaptarlo al de uno mismo. Por otro lado, Render es una plataforma en la nube que permite alojar el servidor donde se gestionan las bases de datos, las rutas API y la lógica del sistema.

4.6 Referencia al repositorio de software

El código fuente del proyecto está disponible en el siguiente repositorio de GitHub:

 <https://github.com/Jorgevf123/AppWeb-TFG.git>

Este repositorio contiene todos los archivos necesarios para ejecutar la aplicación.

La estructura del proyecto está organizada de forma clara, con carpetas como routes, models, public, views y utils, lo que permite que sea más fácil entender el funcionamiento de la plataforma como el sistema de autenticación, la gestión de solicitudes, el mapa de acompañantes, o el panel de administración.

Para ejecutar el proyecto, basta con clonar el repositorio y seguir los pasos indicados en el archivo README.md, donde se explica lo que se debe instalar, comandos a utilizar y la configuración básica del entorno.

Capítulo 5

Resultados

5.1 Plan de pruebas

No se ha utilizado TDD, sino que se ha utilizado pruebas unitarias. Es el usuario el que debe ir validando las cosas y a partir de eso ir avanzando, es decir, es el usuario el que debe ir diciendo si encuentra la aplicación cómoda, si es útil, si cree que satisface sus necesidades, etc.

De acuerdo con las metodologías de gestión de proyectos utilizadas, en un primer momento se fue evaluando de forma individual cada componente de la plataforma, como pueden ser los formularios de solicitud, el sistema de autenticación mediante JWT, etc. Estas pruebas han servido para ver que cada componente responde de manera adecuada antes de implantarlo en el sistema completo. También se han realizado pruebas para asegurar que, si por ejemplo se modifica algo o se envía una solicitud, se actualiza la base de datos. Además, se ha probado el flujo de notificaciones por correo cuando un usuario no está conectado. Por último, se ha comprobado que la aplicación puede manejar varios usuarios conectados a la vez (carga del sistema), asegurando así que los mensajes del chat se envíen y reciban sin errores, y que las solicitudes se gestionen de forma correcta, evitando inconsistencias.

5.2 Aplicación desarrollada

En este punto se pretende dejar claro las operaciones principales de cada tipo de usuario mediante flujos de pantalla, es decir, qué puede y no puede hacer cada uno. Para ello se comenzará explicando el usuario cliente con esta imagen:

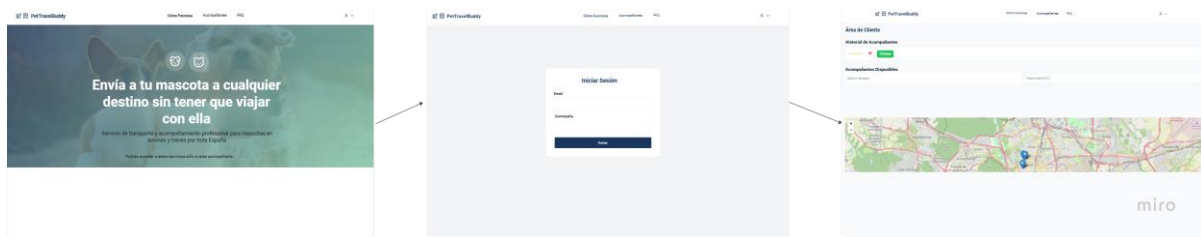


Ilustración 17. Flujo de pantalla cliente. Elaboración propia con Miro

Se puede observar que las acciones más importantes de un cliente son 3. Desde la pantalla principal se podrá mover al inicio de sesión para luego tener acceso al área de cliente donde aparecerá el historial de acompañantes, acompañantes disponibles y un mapa donde salen los acompañantes cercanos. Desde el mismo mapa podrá enviar una solicitud a un acompañante.

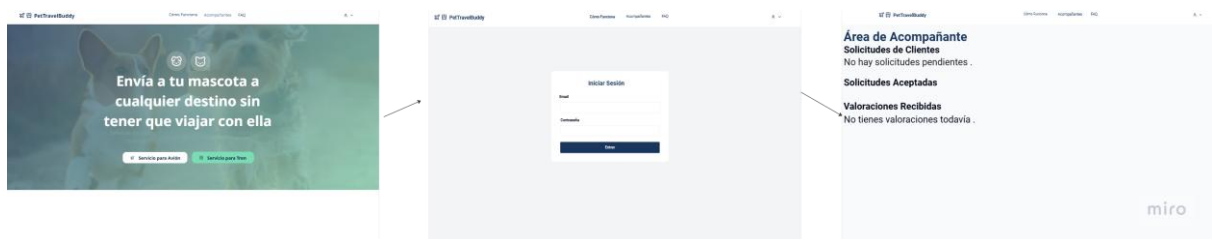


Ilustración 18. Flujo de pantalla acompañante. Elaboración propia con Miro

En este flujo de pantalla, el acompañante tendrá acceso a seleccionar si va a viajar en avión o en tren en la pantalla principal. El inicio de sesión es común para todos y la última pantalla es el área del acompañante donde aparecerán las solicitudes de los distintos clientes, las solicitudes aceptadas y las valoraciones recibidas por parte de los clientes.

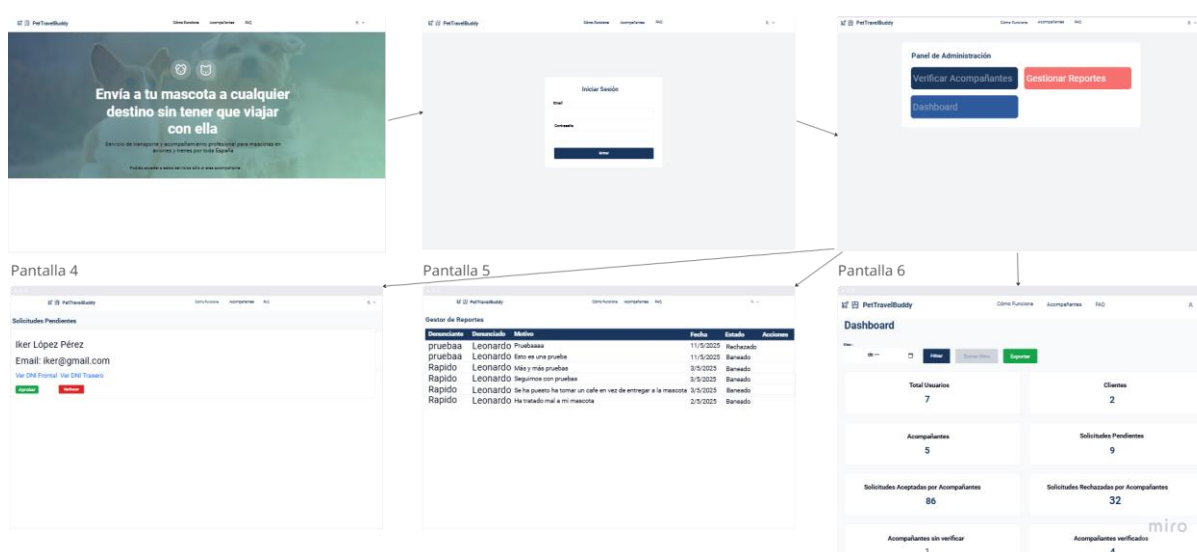


Ilustración 19. Flujo de pantalla administrador. Elaboración propia con Miro

Por último, en este flujo de pantalla el administrador también tiene una pantalla principal y un de inicio de sesión común al resto de roles. Una vez iniciada la sesión, el administrador puede acceder a 3 lugares distintos. El primero es para verificar acompañantes (comprueba que los DNI son reales), en segundo lugar, está la pantalla de bloqueos donde el administrador puede decidir si bloquear o no a los usuarios y por último la pantalla de dashboard donde se verán distintas estadísticas pudiendo filtrar por fecha, así como exportar a un Excel los resultados.

Capítulo 6

Conclusiones y líneas futuras

6.1 Conclusiones

Gracias a la realización de este TFG, he podido aprender distintas cosas, así como enfrentarme a distintos retos. En primer lugar, antes de meterme con el desarrollo de la aplicación web, tuve que hacer un análisis de la competencia, así como hacer uso de distintas herramientas que no conocía como Figma o Draw.io para realizar Wireframes, JourneyMap, User Flows, etc.

En lo que respecta al desarrollo de la aplicación, he aprendido mucho puesto que he tenido que trabajar con MongoDB Atlas y distintos lenguajes como JavaScript, HTML, TypeScript, etc.

Uno de los mayores aprendizajes ha sido la gestión de usuarios y sus distintos roles, comprendiendo la importancia de establecer flujos claros de interacción y asegurando que cada funcionalidad encaje de forma coherente en el sistema total. Para esta parte fue fundamental la realización de JourneyMap comentado anteriormente. También he descubierto lo complejo que puede llegar a ser mantener la sincronización de datos entre las distintas partes de la plataforma.

Por otra parte, está la utilización de distintas APIs y frameworks que no había usado antes y que me han hecho ampliar mi conocimiento en estos campos, como por ejemplo la API OpenRouteService para trazar las rutas de los viajes. Enfrentarme al despliegue ha sido todo un reto ya que nunca había trabajado con Vercel o Render. Me surgieron distintos problemas a lo largo del despliegue mencionado como por ejemplo tener que definir variables de entorno o comprobar que todo funcionara bien, entre otros. Por

tanto, con todo esto doy por satisfecho los distintos objetivos que se tenía desde un principio. El objetivo personal, el cual era aprender cosas nuevas con la realización de este TFG, el objetivo académico de realizar una aplicación web y el objetivo de la aplicación el cual era satisfacer las necesidades de los usuarios haciendo una plataforma útil y confiable.

Por último, me gustaría destacar que el verdadero valor de una plataforma como PetTravelBuddy no reside únicamente en lo técnico, sino en crear una experiencia de usuario útil, segura y eficiente, capaz de generar confianza en un tema tan delicado como el traslado de mascotas.

6.2 Líneas futuras

Aunque la implementación de este TFG haya sido un trabajo duro, es obvio que hay aspectos a mejorar/implementar en un futuro. En primer lugar, uno de los objetivos a futuro es, hacer que el traslado sea lo más real posible y haya un seguimiento de ello. Esto permitiría a los dueños visualizar la ubicación exacta del animal durante el traslado, ofreciendo así una mayor sensación de seguridad y control.

Otro aspecto clave sería integrar un sistema de pago seguro dentro de la plataforma (actualmente es estilo Wallapop). Establecerlo permitiría gestionar los mismos de forma más eficiente.

Por último, creo que sería interesante desarrollar un sistema de valoraciones más detallado, donde los clientes puedan valorar no solo el servicio general, sino también aspectos específicos como la puntualidad, la comunicación, etc.

Creo que estas mejoras no solo harían la plataforma más útil, sino que también ayudarían a aumentar la confianza de los usuarios y tener a PetTravelBuddy como un referente en el traslado seguro de mascotas.

Bibliografía

- [1] Dam, R. F. and Teo, Y. S. (2025, March 15). 6 Common Pitfalls in Prototyping and How to Avoid Them. Interaction Design Foundation - IxDF. <https://www.interaction-design.org/literature/article/prototyping-in-design-thinking-how-to-avoid-six-common-pitfalls>
- [2] ByteByteGo. (2025, February 13). Non-Functional Requirements: The backbone of Great Software - Part 1. ByteByteGo Newsletter. https://blog.bytebytego.com/p/non-functional-requirements-the-backbone?utm_source=substack&utm_medium=email
- [3] ByteByteGo. (2025b, February 20). Non-Functional Requirements: The backbone of Great Software - Part 2. ByteByteGo Newsletter. https://blog.bytebytego.com/p/non-functional-requirements-the-backbone-7db?utm_source=substack&utm_medium=email
- [4] Creativa, A. (2022, April 21). Lo que debes saber sobre la metodología UX | Aula Creativa. Aula Creativa. <https://www.aulacreativa.com/metodologia-ux/>
- [5] Garrett, J. J. (2011). The Elements of User Experience: User-Centered Design for the Web and Beyond (2nd ed.). New Riders.
- [6] Lovable. (n.d.). https://lovable.dev/?via=promos&gad_source=1
- [7] Figma | La herramienta de diseño de interfaz colaborativa. (n.d.). Figma. <https://www.figma.com/es-es/>
- [8] draw.io - free flowchart maker and diagrams online. (n.d.). <https://app.diagrams.net/>
- [9] OOOOU. (2020). Estimate the cost of your app idea. Estimate My App. <https://estimatemyapp.com>
- [10] Miro. (n.d.). Online collaborative whiteboard platform. <https://miro.com/es/>

- [11] OpenAI. (2024). ChatGPT (GPT-4), modelo de lenguaje de IA.
<https://chat.openai.com/>
- [12] EUR-Lex. (2016). Reglamento General de Protección de Datos (UE) 2016/679.
<https://eur-lex.europa.eu/legal-content/ES/TXT/?uri=CELEX%3A32016R0679>

Anexo I – Empathy Map

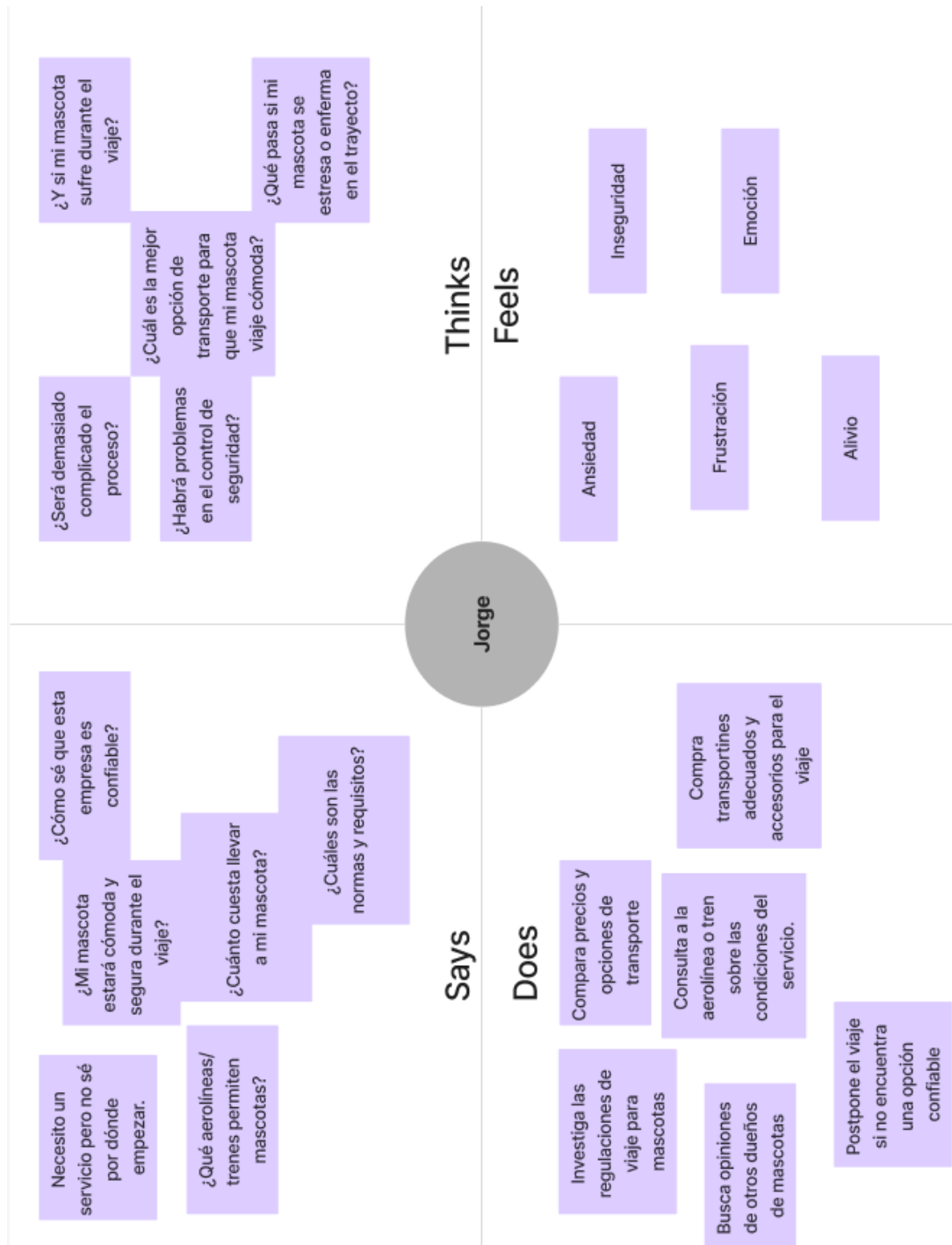


Ilustración 20. Empathy Map. Elaboración propia con Figma

Anexo II – User Persona

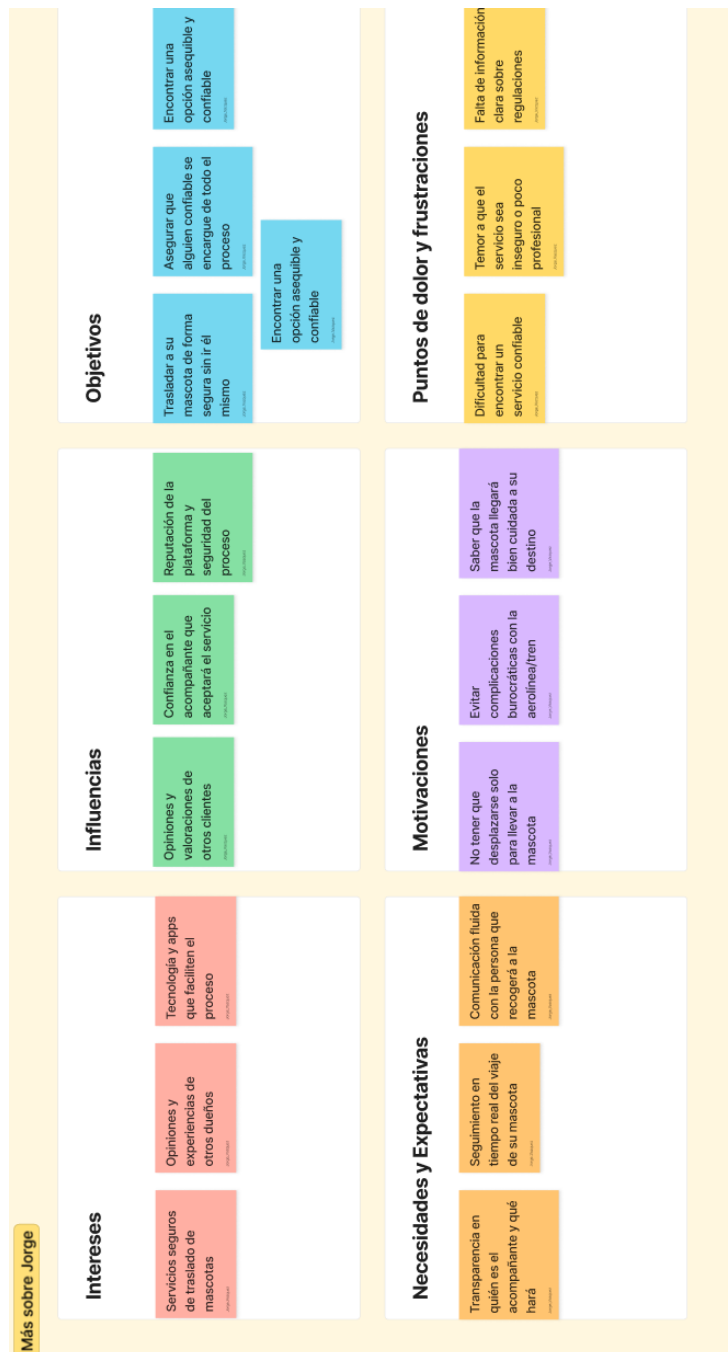


Ilustración 21. Extensión User Persona Jorge

Anexo III – Journey Map

Acción	Buscar servicio de traslado de mascotas	Seleccionar un acompañante disponible	Reservar el servicio	Preparar a la mascota para el viaje	Seguimiento del traslado	Confirmar la llegada al destino
Lista de tareas	A. Abrir la app o web B. Revisar información del servicio C. Leer condiciones del traslado	A. Ver perfiles de acompañantes B. Filtrar por lugar y disponibilidad C. Revisar valoraciones y experiencia	A. Seleccionar un acompañante B. Confirmar horarios y detalles C. Realizar el pago seguro	A. Preparar documentos de la mascota B. Entregar la mascota en el punto de encuentro	A. Recibir notificaciones del estado del traslado (seguimiento en tiempo real) B. Contactar con el acompañante en caso necesario C. Asegurar que todo avanza según lo acordado	A. Recibir la confirmación de llegada B. Verificar que la mascota ha sido entregada C. Dejar una valoración del servicio
Adjetivos de sentimiento	• Ansioso • Dudoso	• Esperanzado • Cauteloso	• Aliviado • Seguro	• Nervioso • Responsable	• Tranquilo • Atento	• Aliviado • Satisfecho
Oportunidades de mejora	• Explicación más clara del servicio • Testimonios de otros usuarios	• Más filtros de búsqueda • Verificación de antecedentes de los acompañantes	• Métodos de pago flexibles • Opción de pago fraccionado	• Lista de requisitos antes del viaje	• GPS en tiempo real • Notificaciones automáticas	• Confirmación con foto • Poder valorar más cosas

Ilustración 22. Journey Map. Elaboración propia con Figma

Anexo IV – Estimación de costes

En este anexo se analizará en más detalle la estimación de costes de la aplicación web (basado en la herramienta Estimate My App):

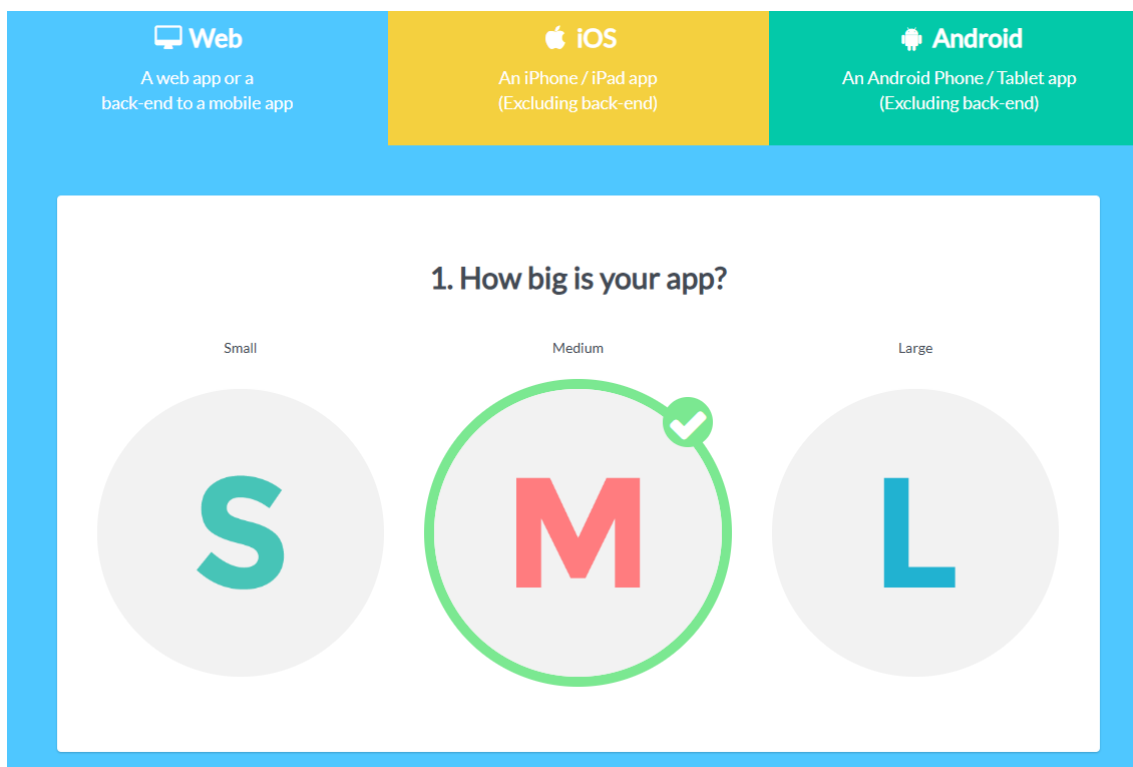


Ilustración 23. Selección del tamaño de la aplicación en la herramienta Estimate My App.

PetTravelBuddy no es una aplicación extremadamente simple como por ejemplo una calculadora, pero tampoco llega a la complejidad de plataformas tipo Uber o redes sociales. Se estima que son unos \$13500 (unos 11900€).

2. What level of UI would you like?

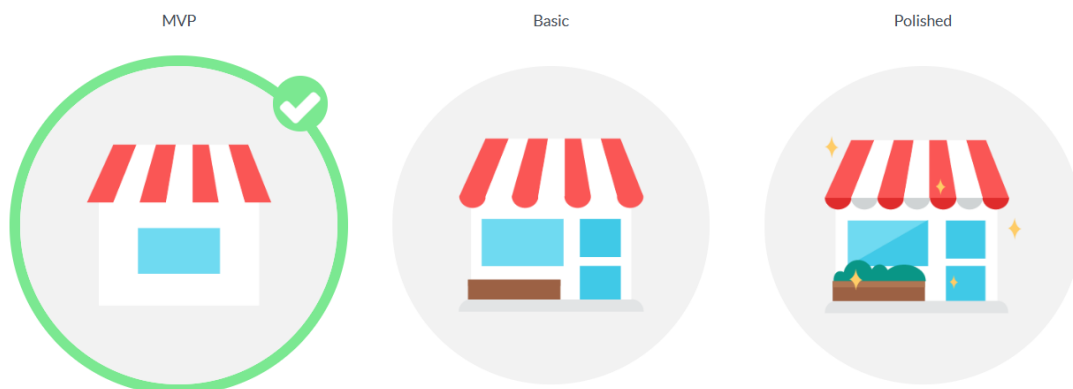


Ilustración 24. Selección nivel interfaz de usuario en la herramienta Estimate My App.

Lo principal es que la aplicación funcione bien, más allá de tener un diseño muy pulido o lleno de animaciones. Se estima un coste de \$4050, unos 3570€.

3. Users & Accounts

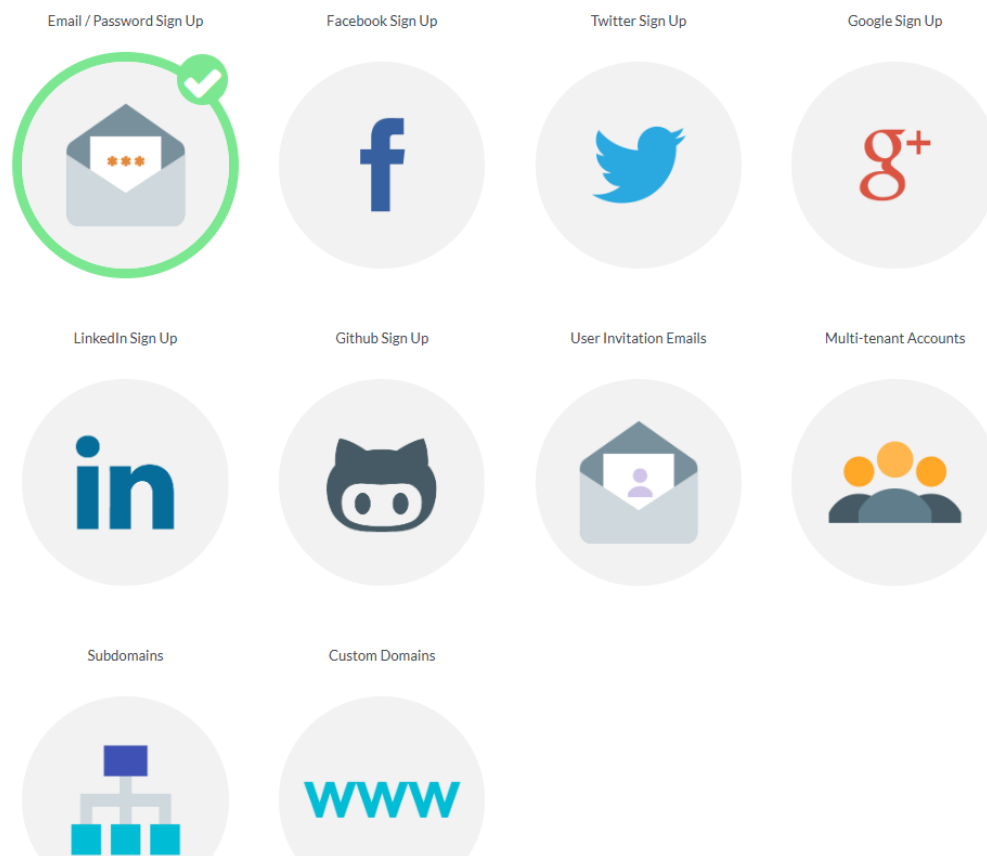


Ilustración 25. Selección usuarios y cuentas en la herramienta Estimate My App.

No se ha implementado el login con redes sociales, únicamente por correo. El coste estimado es de \$450, unos 400€.

4. User Generated Content

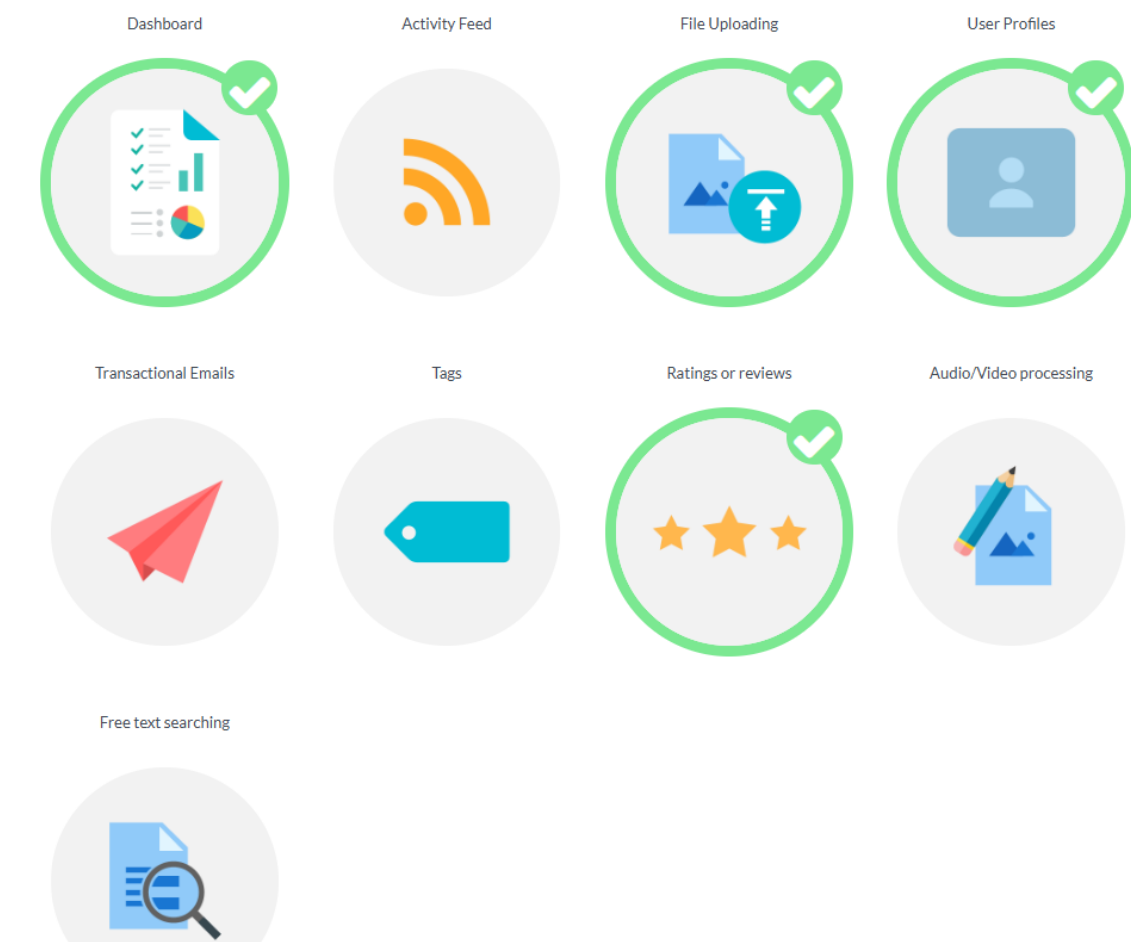
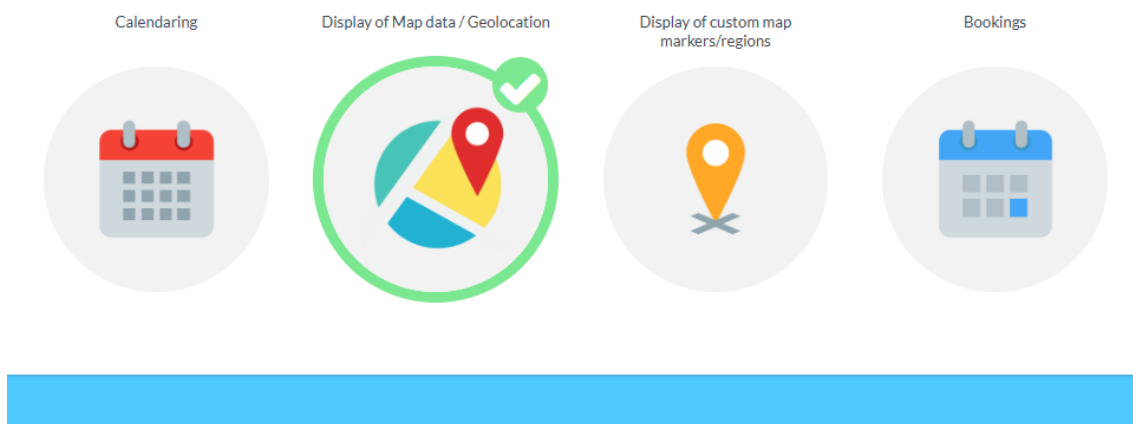


Ilustración 26. Selección contenido generado por el usuario en la herramienta Estimate My App.

El Dashboard está seleccionado ya que el administrador es lo que ve cuando inicia sesión. La subida de archivos también está seleccionada ya que a través del chat entre acompañante y cliente se pueden subir imágenes, ficheros como pdf... Por otro lado, los acompañantes serán vistos por los clientes en la página principal por lo que se ha seleccionado perfiles de usuario. Por último, el cliente podrá valorar al acompañante por su traslado y su trato y es por ello por lo que está seleccionada la opción de reseñas. Todo esto se estima que tiene un coste de \$7200, es decir, unos 6345€.

5. Dates & Locations



6. Social & Engagement

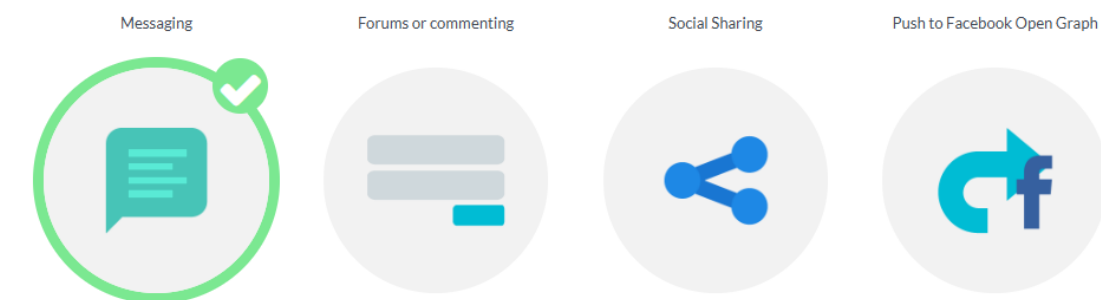


Ilustración 27. Selección localizaciones/fechas y social en la herramienta Estimate My App.

Los clientes ven en un mapa a los acompañantes cercanos y es por ello que está marcado la Geolocalización, además de que al registrarse pide la localización. Por otro lado, está seleccionada la opción de mensajería ya que hay chat entre cliente y acompañante. El coste estimado es de \$4050 que son unos 3570€.

7. Billing & eCommerce

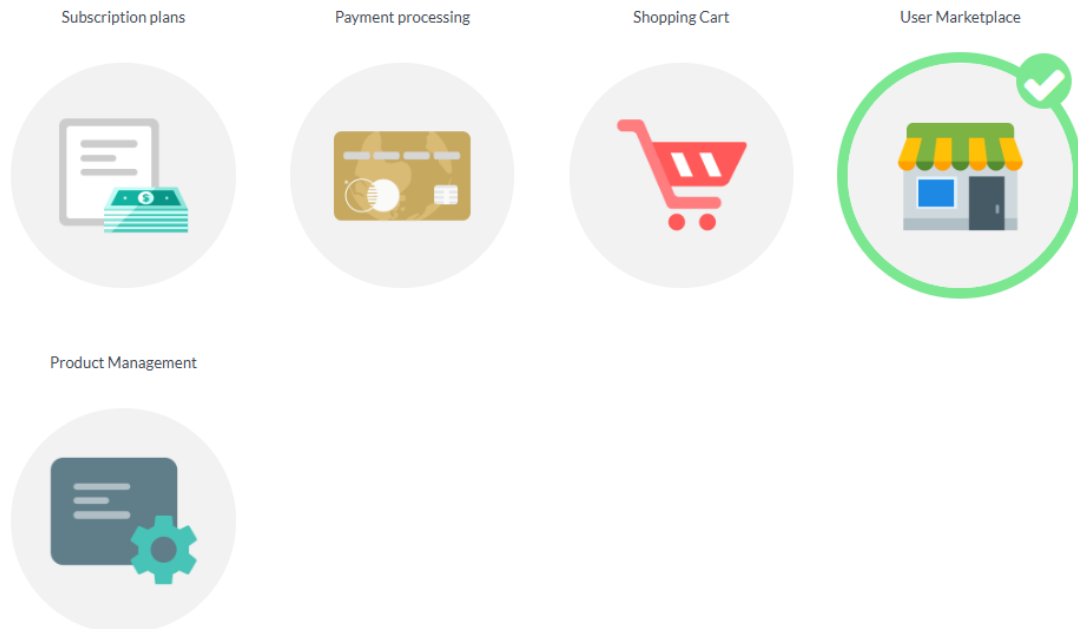


Ilustración 28. Selección facturación y eCommerce en la herramienta Estimate My App.

Está marcada esa opción ya que el sistema conecta a usuarios que ofrecen un servicio con otros que lo solicitan. No hay carrito, suscripciones ni gestión de productos físicos. El método de pago pensado es como el de Wallapop con posibilidad de cambiar a pago con tarjeta en un futuro. El coste estimado es de \$5400, es decir, 4760€.

8. Admin, Feedback & Analytics

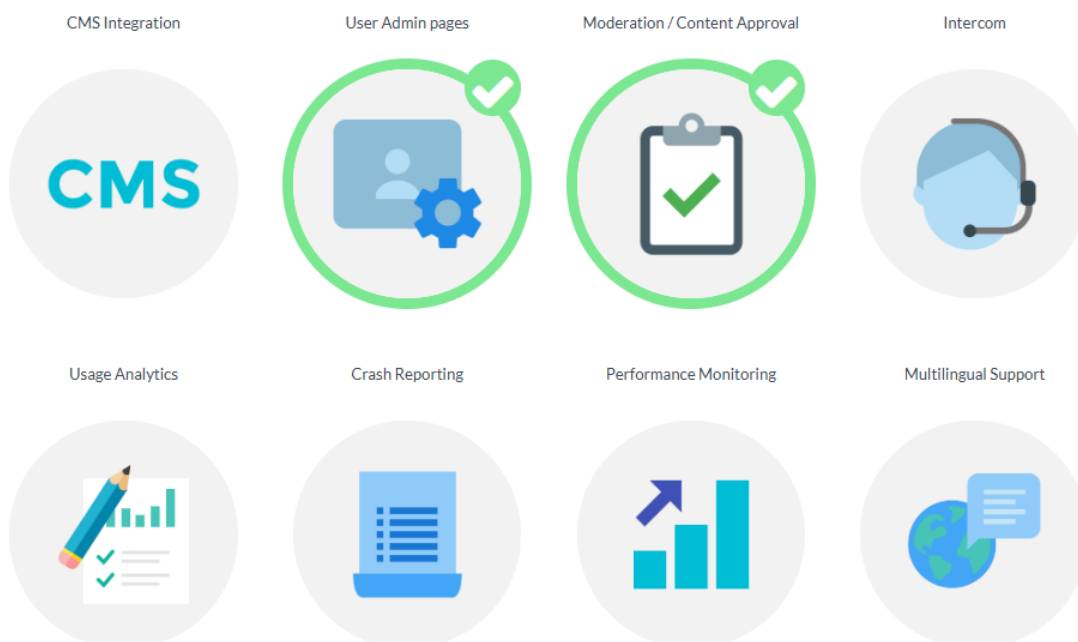


Ilustración 29. Selección admin, feedback y análisis en la herramienta Estimate My App.

Están seleccionadas esas dos opciones debido a que el administrado puede bloquear a usuarios, así como controlar/verificar si por ejemplo el DNI subido por el acompañante es falso o no. Se estima un coste de \$3150, unos 2775€.

9. External APIs and Integrations

Connect to one or more third party services



An API for others to integrate with your app



SMS Messaging



Phone Number Masking



10. Security

SSL Certificate based Security



DoS protection



Two Factor Authentication



Ilustración 30. Selección APIs y seguridad en la herramienta Estimate My App.

En cuanto a las APIs, está marcada esa opción ya que la aplicación depende de varias APIs externas: para geolocalización, rutas, sugerencias de ciudades (Geoapify, ORS) y para el envío de correos (Nodemailer). Por otro lado, en cuanto a la seguridad, aunque no se hayan instalado certificados SSL manualmente, tanto Vercel (despliegue del frontend) como Render (despliegue del backend) proporcionan HTTPS por defecto. El coste estimado es de \$4050, es decir, 3570€.

En resumen, se estima que el total de costes es de \$41850, es decir, 36880€.

En lo que se refiere al blog de Cleveroad, en general, el coste de creación de una aplicación oscila entre \$ 40,000 y \$ 300,000. Pero, si se pregunta a los proveedores de varios países, darán diferentes números. Esto se debe a las grandes diferencias en las tarifas por hora según la ubicación de la empresa de software.

Región	iOS (\$)	Android (\$)
América del Norte	\$120-\$150	\$120-\$150
Reino Unido	\$70-\$80	\$70-\$80
Europa del Norte y del Este	\$45-\$60	\$45-\$60
Australia	\$100-\$120	\$100-\$120
América del Sur	\$40-\$45	\$35-\$40
India	\$20-\$25	\$21-\$26

Ilustración 31. Rango de precios por hora estimado para el desarrollo de aplicaciones según Cleveroad (2024)

Echando un vistazo a cómo difiere el coste promedio del desarrollo de aplicaciones en varias partes del mundo, es decir, cuánto cuesta hacer una aplicación en diferentes partes del mundo, se aprecia que la tarifa por hora de un desarrollador de América del Norte es muy superior a la de la India tanto en iOS como en Android. Esto se debe en gran parte a factores como la experiencia del equipo de desarrollo, tecnologías utilizadas, ya que algunas tecnologías requieren desarrolladores más especializados o escasos en el mercado y conocimiento del dominio, puesto que si una app pertenece a un sector específico puede requerir conocimientos adicionales que también afectan al precio.

Anexo V – Wireframe

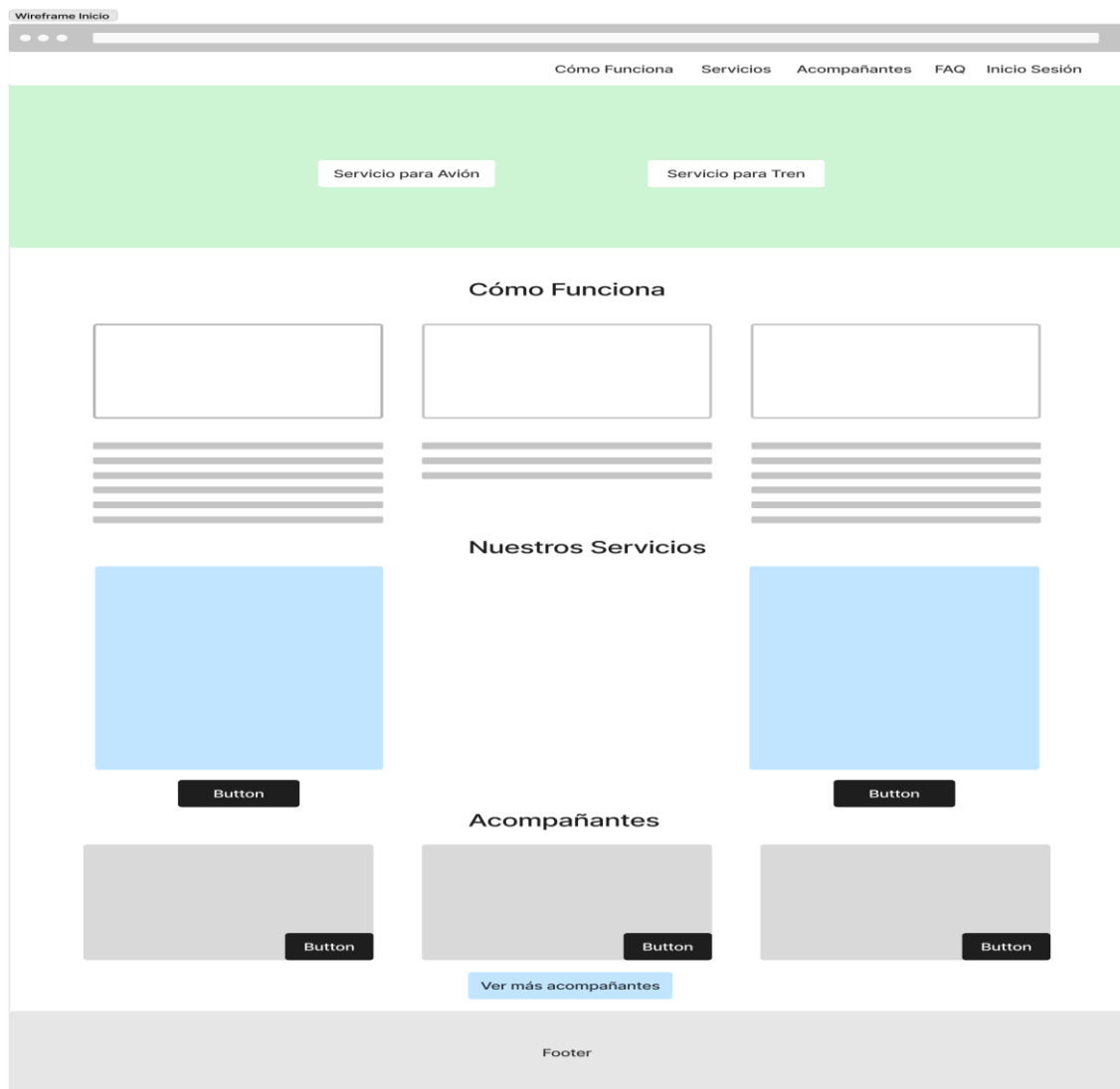


Ilustración 32. Wireframe Inicio. Elaboración propia con Figma.

Anexo VI – Colecciones

```
_id: ObjectId('68335959ac8957112704e58c')
clienteId: ObjectId('67f956a280b034b9f39bedeb')
acompananteId: ObjectId('680e5b5a893b03309d654f5a')
tipoAnimal: "ave"
raza: "Loro"
dimensiones: "3x2x1"
vacunasAlDia: false
estado: "aceptada"
matchId: ObjectId('68335959ac8957112704e58a')
valoracionPendiente: false
fechaSolicitud: 2025-05-25T17:54:33.181+00:00
__v: 0
```

Ilustración 33. Documento de ejemplo de la colección solicitudes en MongoDB.

Esta colección es la de las solicitudes, es decir, cuando el cliente rellena el formulario de tipo de animal, raza, dimensiones y vacunas al día a la hora de solicitar a un acompañante. Más adelante y una vez completado el trayecto, el usuario podrá valorar al acompañante por ese viaje en concreto y es por eso por lo que está a false.

```
_id: ObjectId('68207b4550dbb44aafa8d70b')
remiteinte: ObjectId('67f956a280b034b9f39bedeb')
destinatario: ObjectId('680e5b5a893b03309d654f5a')
motivo: "Esto es una prueba"
estado: "baneado"
fecha: 2025-05-11T10:26:13.398+00:00
__v: 0
```

Ilustración 34. Documento de ejemplo de la colección reportes en MongoDB.

Esta colección es la de reportes, en la cual un usuario puede reportar a un acompañante dando el motivo. Es el administrador el que se encarga de bloquearlo o no y en caso

afirmativo, se pone un tiempo. La persona bloqueada no podrá iniciar sesión durante todo ese tiempo de bloqueo.

```
_id: ObjectId('683356c3ac8957112704e363')  
clienteId: ObjectId('67f956a280b034b9f39bedeb')  
acompananteId: ObjectId('680e5b5a893b03309d654f5a')  
estado: "completado"  
finalizado: true  
fecha: 2025-05-25T17:43:31.949+00:00  
__v: 0  
comentarioCliente: "Todo bien"  
valoracionCliente: 5
```

Ilustración 35. Documento de ejemplo de la colección matches en MongoDB.

Esta colección es la de los matches, es decir, cuando un acompañante acepta la solicitud enviada por un cliente. También aparecerá aquí lo que el cliente le ha valorado (estrellas y comentario).

```
_id: ObjectId('683f3dd6de2499dbb6114614')  
participantes: Array (2)  
  0: ObjectId('680e5b5a893b03309d654f5a')  
  1: ObjectId('67f956a280b034b9f39bedeb')  
matchId: ObjectId('67f956a280b034b9f39bedeb')  
__v: 0
```

Ilustración 36. Documento de ejemplo de la colección chats en MongoDB.

Esta colección es la de chats. En ella se encuentran los usuarios que participan en el chat (siempre serán 2).

```
_id: ObjectId('683f3dd6de2499dbb6114616')
chatId: ObjectId('683f3dd6de2499dbb6114614')
remitente: ObjectId('680e5b5a893b03309d654f5a')
texto: "Si?"
timestamp: 2025-06-03T18:24:22.598+00:00
__v: 0
```

```
_id: ObjectId('683f3de2de2499dbb611461b')
chatId: ObjectId('683f3d28de2499dbb611447a')
remitente: ObjectId('67f956a280b034b9f39bedeb')
texto: ""
archivo: "1748975074621-R02 - 3.3 Voz sobre IP.pdf"
timestamp: 2025-06-03T18:24:34.782+00:00
__v: 0
```

Ilustración 37. Documento de ejemplo de la colección mensajes en MongoDB

La última colección es la de mensajes. Aquí cada mensaje se guarda en un documento individual. Tiene un chatId asociado que debería coincidir con el id de la colección chats para indicar entre que dos participantes está teniendo lugar la conversación.